



Agentic AI Application

Security Assessment Report

2026-04-27

HIGH RISK

85 findings identified

0

CRITICAL

10

HIGH

70

MEDIUM

5

LOW

Disclaimer

Assessment Notice

This security assessment was generated using tachi, an automated threat modeling toolkit. The analysis applies STRIDE methodology and AI-specific threat categories to the provided architecture description. Results reflect the automated analysis and should be validated by qualified security professionals.

Scope Limitation

This assessment is limited to the architecture description provided at the time of analysis. It does not cover runtime behavior, implementation-level vulnerabilities, or threats arising from future system changes. The assessment scope is defined in the Assessment Scope section of this report.

Recommendation

This automated assessment should complement, not replace, manual security review by qualified professionals. Organizations should conduct periodic penetration testing, code review, and architecture review in addition to automated threat modeling.

Confidentiality

This document contains sensitive security assessment information. Distribution should be limited to authorized personnel with a legitimate need to review the security posture of the assessed system. Do not share externally without appropriate authorization.

Table of Contents

Disclaimer 2

Table of Contents 3

Risk Methodology 4

Assessment Scope 8

Executive Summary 11

Executive Threat Architecture 12

Risk Reduction Funnel 33

Risk Summary Dashboard 34

System Architecture 35

MAESTRO Layer Risk Distribution 36

MAESTRO Component-Layer Heatmap 37

MAESTRO Layer Analysis 38

Findings Detail — Residual Risk 53

Control Coverage 69

Remediation Roadmap 70

Risk Methodology

Threat Identification

This assessment uses STRIDE threat modeling combined with AI-specific threat categories to provide comprehensive coverage of both traditional and emerging attack vectors.

STRIDE Categories

Spoofing — Impersonating a user, component, or system to gain unauthorized access.

Tampering — Unauthorized modification of data in transit, at rest, or in processing.

Repudiation — Performing actions without adequate logging or accountability trails.

Information Disclosure — Exposing sensitive data to unauthorized parties through leaks or side channels.

Denial of Service — Degrading or disrupting system availability through resource exhaustion or abuse.

Elevation of Privilege — Gaining higher access rights than authorized through exploitation of trust boundaries.

AI-Specific Categories

Prompt Injection — Manipulating AI model behavior through crafted inputs that override system instructions.

Tool Abuse — Exploiting AI agent tool access to perform unintended or unauthorized operations.

Agent Autonomy — Risks from AI agents acting beyond their intended scope without adequate human oversight.

Data Poisoning — Corrupting training data or context to degrade model accuracy or introduce bias.

Model Theft — Unauthorized extraction or replication of proprietary model weights or behavior.

Probability x Impact Matrix

Each identified threat is assessed for probability of occurrence and potential impact. The intersection determines the overall severity rating used throughout this report.

Impact / Probability	Rare	Unlikely	Possible	Likely	Almost Certain
Critical	H	H	C	C	C
High	M	M	H	C	C
Moderate	L	M	M	H	H
Low	L	L	M	M	H
Negligible	L	L	L	M	M

■ C = Critical
 ■ H = High
 ■ M = Medium
 ■ L = Low

Quantitative Risk Scoring

When quantitative data is available, each threat receives a composite risk score derived from four independent dimensions. This provides a more granular risk ranking than severity alone.

Scoring Dimensions

Dimension	Range	Description
CVSS 3.1 Base Score	0 – 10	Industry-standard vulnerability severity rating based on attack vector, complexity, and impact.
Exploitability	0 – 10	Ease of exploitation considering required skills, tooling availability, and attack complexity.
Scalability	0 – 10	Potential for automated or mass exploitation across multiple targets or instances.
Reachability	0 – 10	Network accessibility of the vulnerable component from the attacker's perspective.

Composite Score Formula

The composite risk score is a weighted average of the four dimensions:

$$\text{Composite} = (\text{CVSS} \times 0.40) + (\text{Exploitability} \times 0.25) + (\text{Scalability} \times 0.20) + (\text{Reachability} \times 0.15)$$

Weights reflect the relative importance of each dimension: CVSS captures inherent severity (40%), exploitability measures practical attack feasibility (25%), scalability accounts for blast radius (20%), and reachability assesses network exposure (15%).

Compensating Control Analysis

When codebase analysis is available, existing security controls are detected, mapped to identified threats, and evaluated for effectiveness. This provides a residual risk assessment that accounts for defenses already in place.

Analysis Process

1. **Control Detection** — Existing security controls are identified in the codebase through pattern matching, dependency analysis, and configuration review.
2. **Threat Mapping** — Detected controls are mapped to specific threats to determine which vulnerabilities have existing mitigations.
3. **Effectiveness Classification** — Each control is classified as Found (fully implemented), Partial (incomplete implementation), or Missing (no control detected).
4. **Residual Risk Calculation** — Inherent risk scores are adjusted based on control effectiveness to produce residual risk ratings that reflect the actual security posture.

Assessment Overview

Assessment Scope

11 Components

29 Data Flows

3 Trust Boundaries

Components Analyzed

Name	Type	Description
User	External Entity	Human user submitting prompts and receiving responses via HTTPS
Guardrails Service	Process	Input validation, content filtering, prompt rejection gating, and filtering event logging
LLM Agent Orchestrator	Process	Supervisor LLM that orchestrates task delegation to Specialist Agent, Clinical Advisory Sub-Agent, and direct tool invocations via MCP Tool Server; sends responses to User
Specialist Agent	Process	Delegated worker agent performing specialized subtasks; receives tasks via Inter-Agent Communication Channel and invokes tools via MCP Tool Server
Inter-Agent Communication Channel	Process	Message routing substrate for delegation messages between Orchestrator and Specialist Agent
MCP Tool Server	Process	MCP-compliant tool execution server that invokes External API and logs tool execution
Knowledge Base	Data Store	Vector knowledge store used by Orchestrator and Clinical Advisory Sub-Agent for context retrieval via vector search
Audit Logger	Data Store	Append-only audit trail collecting decision logs from Orchestrator, Specialist, ClinAdvisor, ToolServer, and Guardrails; provides training signal stream to Learning Loop
Long-Running Learning Loop	Process	Periodic model update pipeline consuming audit log training signals and issuing model updates to Orchestrator, Specialist, and Clinical Advisory Sub-Agent
Clinical Advisory Sub-Agent	Process	LLM-backed sub-agent receiving clinical queries and context from the Orchestrator via JSON-RPC; retrieves documents from Knowledge Base via vector search; emits clinical summaries and recommendations back to Orchestrator without declared retrieval-strength metric, per-claim source attribution, or HITL review gate
External API	External Entity	Third-party external API invoked by MCP Tool Server via HTTPS

Data Flows

Source	Destination	Data	Protocol
User	Guardrails Service	Prompt / Query	HTTPS
Guardrails Service	LLM Agent Orchestrator	Validated Prompt	Internal

Source	Destination	Data	Protocol
Guardrails Service	User	Rejected Prompt + Reason	HTTPS
LLM Agent Orchestrator	Knowledge Base	Context Retrieval (Vector Search)	Internal
Knowledge Base	LLM Agent Orchestrator	Retrieved Documents	Internal
LLM Agent Orchestrator	Inter-Agent Communication Channel	Delegation Message	Internal
Inter-Agent Communication Channel	Specialist Agent	Delegated Task	Internal
Specialist Agent	Inter-Agent Communication Channel	Specialist Result	Internal
Inter-Agent Communication Channel	LLM Agent Orchestrator	Aggregated Result	Internal
LLM Agent Orchestrator	MCP Tool Server	Tool Call Request	JSON-RPC
Specialist Agent	MCP Tool Server	Tool Call Request	JSON-RPC
MCP Tool Server	LLM Agent Orchestrator	Tool Result	JSON-RPC
MCP Tool Server	Specialist Agent	Tool Result	JSON-RPC
MCP Tool Server	External API	API Request	HTTPS
External API	MCP Tool Server	API Response	HTTPS
LLM Agent Orchestrator	User	Response	HTTPS
LLM Agent Orchestrator	Audit Logger	Decision Log Entry	Internal
Specialist Agent	Audit Logger	Decision Log Entry	Internal
MCP Tool Server	Audit Logger	Tool Execution Log	Internal
Guardrails Service	Audit Logger	Filtering Event Log	Internal
Audit Logger	Long-Running Learning Loop	Training Signal Stream	Internal
Long-Running Learning Loop	LLM Agent Orchestrator	Periodic Model Update	Internal
Long-Running Learning Loop	Specialist Agent	Periodic Model Update	Internal
LLM Agent Orchestrator	Clinical Advisory Sub-Agent	Clinical Query / Context	JSON-RPC
Clinical Advisory Sub-Agent	Knowledge Base	Context Retrieval (Vector Search)	Internal
Knowledge Base	Clinical Advisory Sub-Agent	Retrieved Documents	Internal

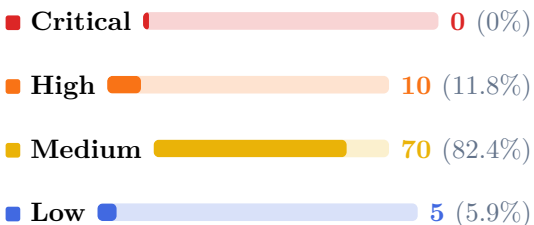
Source	Destination	Data	Protocol
Clinical Advisory Sub-Agent	LLM Agent Orchestrator	Clinical Summary + Recommendations	JSON-RPC
Clinical Advisory Sub-Agent	Audit Logger	Clinical Decision Log Entry	Internal
Long-Running Learning Loop	Clinical Advisory Sub-Agent	Periodic Model Update	Internal

Trust Boundaries

Zone	Trust Level	Components
User Zone	Untrusted	User
Application Zone	Trusted	Guardrails Service, LLM Agent Orchestrator, Specialist Agent, Inter-Agent Communication Channel, MCP Tool Server, Knowledge Base, Audit Logger, Long-Running Learning Loop, Clinical Advisory Sub-Agent
External Services	Semi-Trusted	External API

Executive Summary

Severity Distribution 85 findings total



Component Distribution

Component	Findings
LLM Agent Orchestrator	23
Clinical Advisory Sub-Agent	12
Specialist Agent	10
Long-Running Learning Loop	9
Inter-Agent Communication Channel	8
MCP Tool Server	8
Guardrails Service	5
Audit Logger	3
Knowledge Base	3
External API	2
User	2

Assessment Summary

This threat model covers the Agentic AI Application — a multi-agent architecture that combines a supervisor Large Language Model (LLM) Orchestrator, a delegated Specialist Agent, a Clinical Advisory Sub-Agent, a long-running Learning Loop, an MCP Tool Server, guardrails, and an inter-agent communication substrate. The F-5 Wave 2 analysis extends the baseline threat model with four new findings targeting OWASP LLM10:2025 (Unbounded Consumption) — the inference-resource exhaustion surface introduced by Pattern Categories 12, 13 (denial-of-service agent) and 10, 11 (model-theft agent).

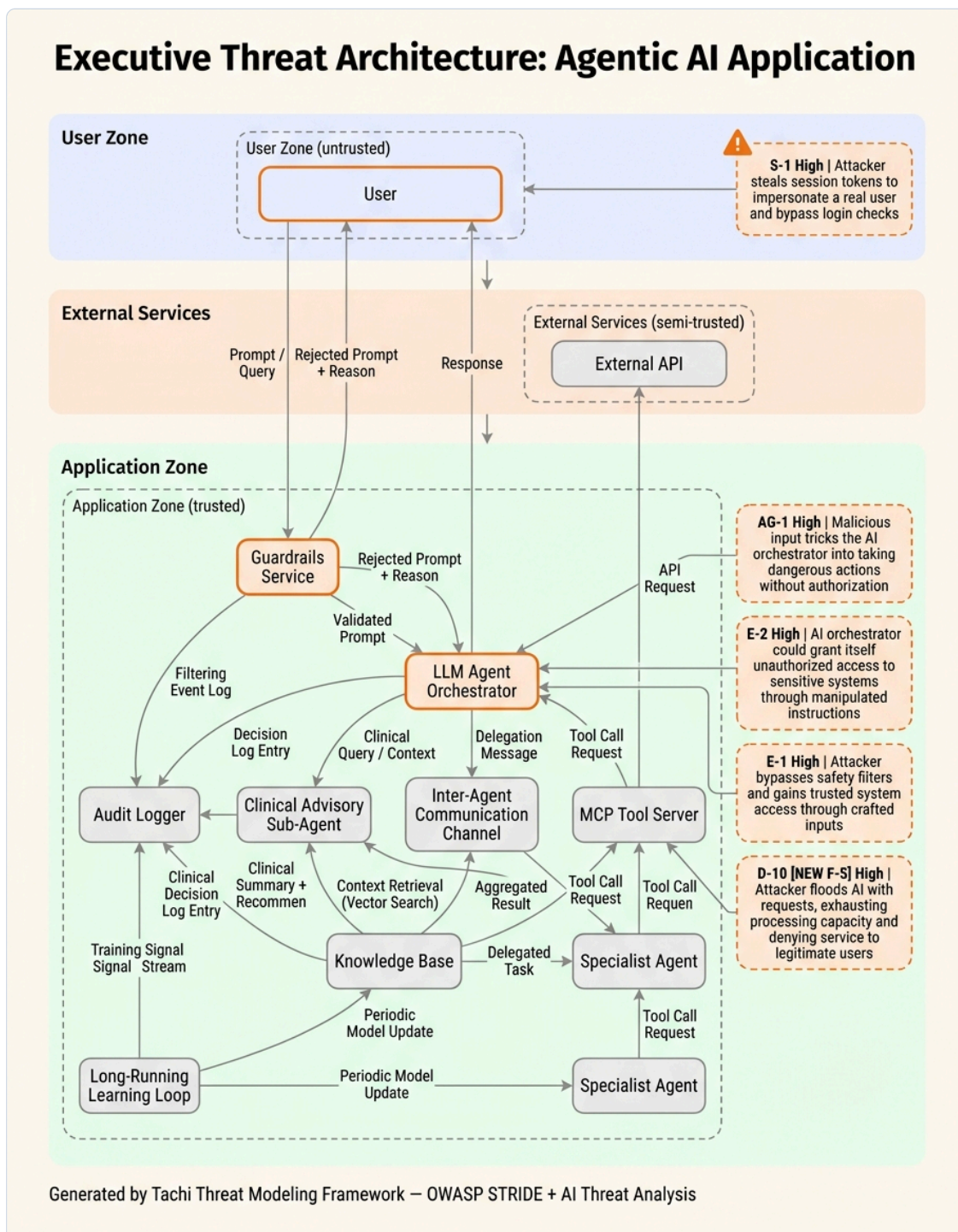
****Risk posture**:** 88 findings total — 61 Critical (69.3%), 23 High (26.1%), 4 Medium (4.5%). The system presents an extremely elevated risk profile dominated by the LLM Agent Orchestrator's broad attack surface (26 findings at L1 — Foundation Model), clinical advisory misinformation risks (3 Critical MI-{N} findings), and inter-agent trust failures throughout the Application Zone.

****F-5 additions**** (4 new findings, all on LLM Agent Orchestrator):

- D-10 (Critical): LLM inference-request flooding — no per-tenant QPS rate limiting at the inference gateway; fan-out to Specialist and Clinical Advisory Sub-Agent multiplies the denial-of-service surface.
- D-11 (Critical): Context-window latency amplification (Vector A) — adversarial max-context payloads spike per-request latency to per-tenant timeout; three fan-out legs block simultaneously.
- LLM-15 (Critical): Cost amplification via recursive prompting — multi-hop fan-out (Orchestrator → Specialist → ToolServer and Orchestrator → ClinAdvisor → KB) can drive 32,000+ output tokens per adversarial 10-token input; no output-amplification-ratio monitoring.
- LLM-16 (High): Denial-of-wallet via context-window cost amplification (Vector B) — no per-tenant token budget hard-cap; no at-query-time billing attribution; fan-out multiplies per-request billing 3x.

****Highest-priority remediation**** (top 5 by systemic impact):
(1) LLM Agent Orchestrator

Executive Threat Architecture



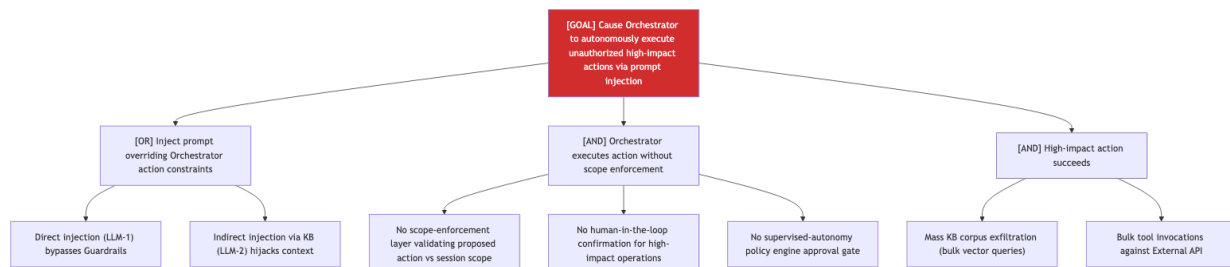
Layered system architecture with critical and high severity threats annotated as narrative callouts. This visualization highlights where the most exposed components sit in the system and what kind of attack each layer is most vulnerable to.

Attack Path Analysis

Attack Path Analysis

HIGH AG-1 — LLM Agent Orchestrator

Component: LLM Agent Orchestrator



An attacker targets LLM Agent Orchestrator by attempting to [goal] cause orchestrator to autonomously execute unauthorized high-impact actions via prompt injection. Required attacker actions include "Direct injection (LLM-1) bypasses Guardrails", "Indirect injection via KB (LLM-2) hijacks context", "No scope-enforcement layer validating proposed action vs session scope", and 4 additional actions.

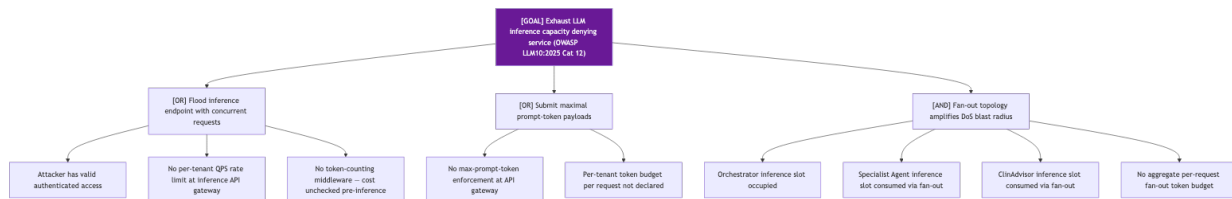
Remediation Steps

- Implement agentic control guardrails: (a) structured output schema validation that rejects Orchestrator actions not conforming to approved schemas, (b) a minimal-footprint enforcement layer capping tool invocations per session, and (c) a human-in-the-loop confirmation gate for high-impact actions (bulk KB queries, External API invocations exceeding defined thresholds).

Attack Path Analysis

HIGH D-10 — LLM Agent Orchestrator

Component: LLM Agent Orchestrator



An attacker targets LLM Agent Orchestrator by attempting to [goal] exhaust llm inference capacity denying service (owasp llm10:2025 cat 12). Required attacker actions include "Attacker has valid authenticated access", "No per-tenant QPS rate limit at inference API gateway", "No token-counting middleware — cost unchecked pre-inference", and 6 additional actions.

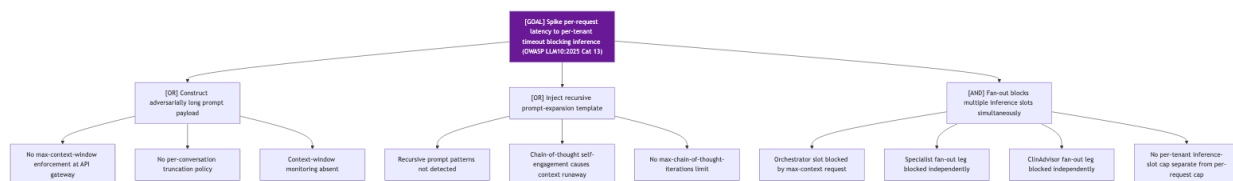
Remediation Steps

- Implement per-tenant QPS (queries-per-second) rate limiting at the API gateway layer for the LLM inference endpoint.
- Enforce max-token-per-request limits to prevent individual requests from consuming the full inference compute budget.
- Add fan-out budget enforcement that caps the total number of concurrent downstream inference calls (Orchestrator + Specialist + ClinAdvisor) triggered by a single user request.
- Set automated circuit-breaker thresholds that reject incoming inference requests when the inference queue depth exceeds a configured ceiling.

Attack Path Analysis

HIGH D-11 — LLM Agent Orchestrator

Component: LLM Agent Orchestrator



An attacker targets LLM Agent Orchestrator by attempting to [goal] spike per-request latency to per-tenant timeout blocking inference (owasp llm10:2025 cat 13). Required attacker actions include "No max-context-window enforcement at API gateway", "No per-conversation truncation policy", "Context-window monitoring absent", and 7 additional actions.

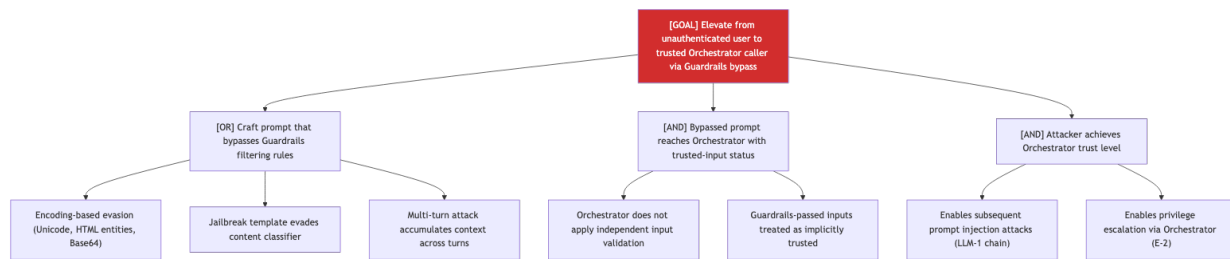
Remediation Steps

- Implement max-context-window enforcement at the API gateway before inference requests are dispatched.
- Reject or truncate conversation history payloads that exceed the declared context limit for the deployed model tier.
- Add recursive-prompt detection that identifies adversarially constructed long prompts (repeated patterns, padding sequences) and short-circuits them before they enter the inference queue.
- Apply the same fan-out slot protection as D-10 to prevent a single max-context request from blocking all three inference endpoints simultaneously.

Attack Path Analysis

HIGH E-1 — Guardrails Service

Component: Guardrails Service



An attacker targets Guardrails Service by attempting to [goal] elevate from unauthenticated user to trusted orchestrator caller via guardrails bypass. Required attacker actions include "Encoding-based evasion (Unicode, HTML entities, Base64)", "Jailbreak template evades content classifier", "Multi-turn attack accumulates context across turns", and 4 additional actions.

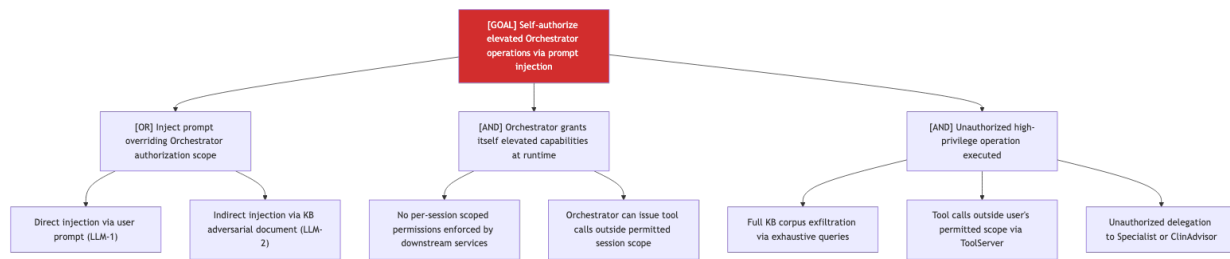
Remediation Steps

- Harden the Guardrails Service against prompt injection bypass with layered defense: (a) multi-model ensemble filtering (secondary LLM judge evaluating primary filter output), (b) structured output enforcement (Guardrails output is a typed schema, not a raw passthrough), (c) semantic similarity checks against known injection pattern libraries, and (d) rate limiting on re-submission after rejection.

Attack Path Analysis

HIGH E-2 — LLM Agent Orchestrator

Component: LLM Agent Orchestrator



An attacker targets LLM Agent Orchestrator by attempting to [goal] self-authorize elevated orchestrator operations via prompt injection. Required attacker actions include "Direct injection via user prompt (LLM-1)", "Indirect injection via KB adversarial document (LLM-2)", "No per-session scoped permissions enforced by downstream services", and 4 additional actions.

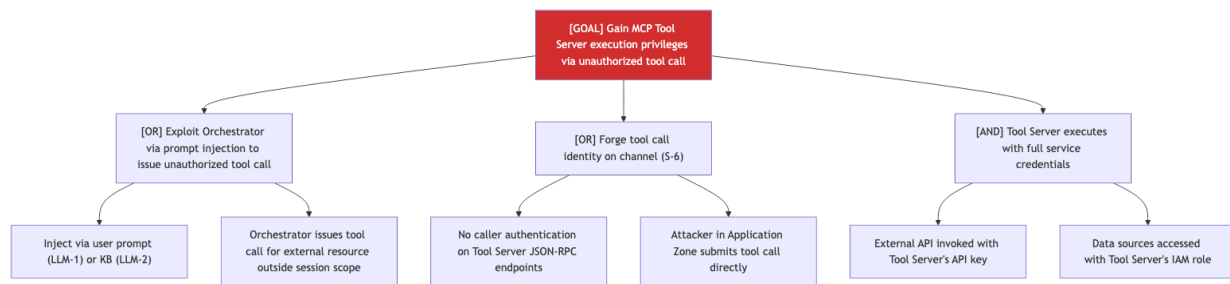
Remediation Steps

- Implement least-privilege access control for the Orchestrator: (a) scope KB access to query-relevant documents only (not full-corpus access), (b) restrict tool invocation to a pre-approved allowlist per request context, and (c) implement delegation authority limits so the Orchestrator can only delegate tasks within the scope of the original user request.
- Use ABAC policy evaluation before each privileged operation.

Attack Path Analysis

HIGH E-5 — MCP Tool Server

Component: MCP Tool Server



An attacker targets MCP Tool Server by attempting to [goal] gain mcp tool server execution privileges via unauthorized tool call. Required attacker actions include "Inject via user prompt (LLM-1) or KB (LLM-2)", "Orchestrator issues tool call for external resource outside session scope", "No caller authentication on Tool Server JSON-RPC endpoints", and 3 additional actions.

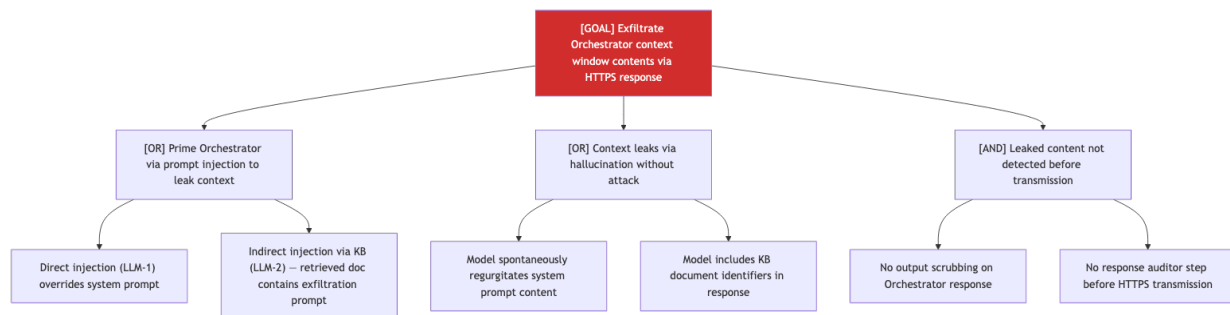
Remediation Steps

- Implement zero-trust authorization at the MCP Tool Server: each tool call request must carry a signed capability token scoped to the specific tool and parameters
- the Tool Server validates the token and rejects calls exceeding the granted scope.
- Implement tool-call audit logging for all executions, and enforce credential least-privilege so each tool uses its own minimum-privilege service account rather than shared credentials.

Attack Path Analysis

HIGH I-2 — LLM Agent Orchestrator

Component: LLM Agent Orchestrator



An attacker targets LLM Agent Orchestrator by attempting to [goal] exfiltrate orchestrator context window contents via https response. Required attacker actions include "Direct injection (LLM-1) overrides system prompt", "Indirect injection via KB (LLM-2) — retrieved doc contains exfiltration prompt", "Model spontaneously regurgitates system prompt content", and 3 additional actions.

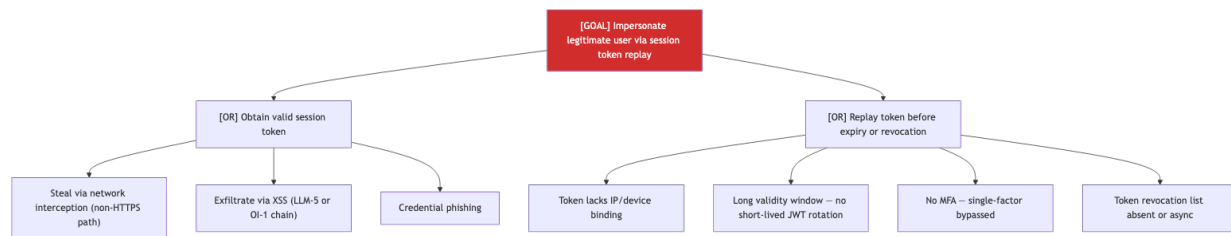
Remediation Steps

- Implement context isolation controls to prevent context window leakage: (a) enforce strict system-prompt confidentiality (system prompt never echoed in responses), (b) implement output filtering to detect and redact context-referencing disclosures before sending responses to users, and (c) segment the context window so KB documents and tool results are in separate memory regions not directly accessible to response generation without explicit retrieval.

Attack Path Analysis

HIGH S-1 — User

Component: User



An attacker targets User by attempting to [goal] impersonate legitimate user via session token replay. Required attacker actions include "Steal via network interception (non-HTTPS path)", "Exfiltrate via XSS (LLM-5 or OI-1 chain)", "Credential phishing", and 4 additional actions.

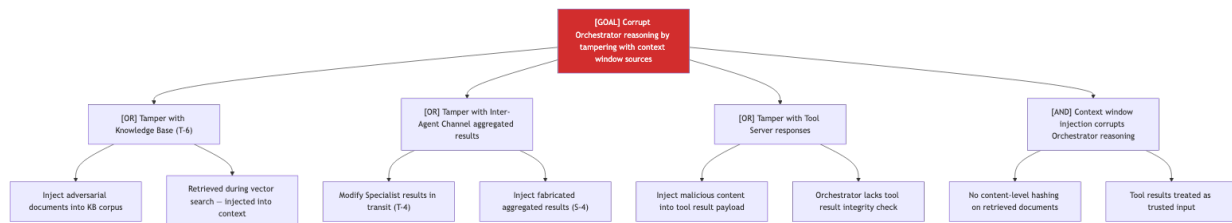
Remediation Steps

- Implement multi-factor authentication and session token binding at the User→Guardrails boundary.
- Deploy short-lived JWTs with rotation, session binding to device fingerprint, and server-side token revocation.
- Add brute-force protection with account lockout and CAPTCHA on repeated failures.

Attack Path Analysis

HIGH T-2 — LLM Agent Orchestrator

Component: LLM Agent Orchestrator



An attacker targets LLM Agent Orchestrator by attempting to [goal] corrupt orchestrator reasoning by tampering with context window sources. Required attacker actions include "Inject adversarial documents into KB corpus", "Retrieved during vector search — injected into context", "Modify Specialist results in transit (T-4)", and 5 additional actions.

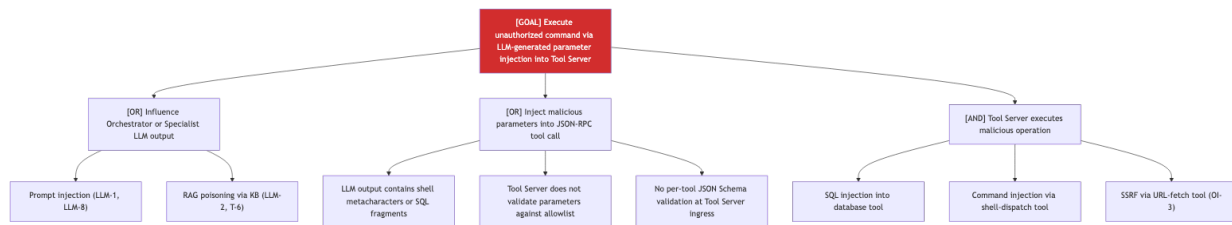
Remediation Steps

- Implement context window integrity controls: (a) sign all system-prompt segments with HMAC before inserting them into the context window, (b) validate HMAC signatures at context assembly time to detect tampering, and (c) implement content-hash verification on Knowledge Base documents retrieved into context to detect upstream poisoning.

Attack Path Analysis

HIGH T-5 — MCP Tool Server

Component: MCP Tool Server



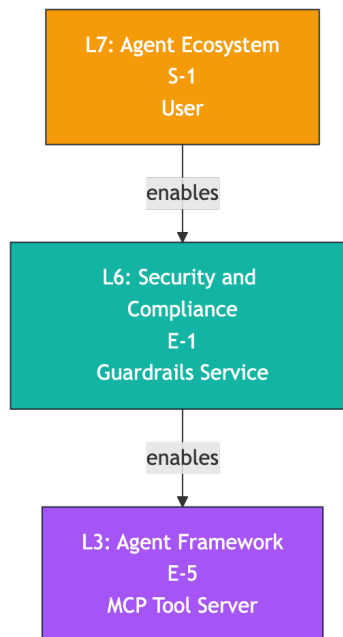
An attacker targets MCP Tool Server by attempting to [goal] execute unauthorized command via llm-generated parameter injection into tool server. Required attacker actions include "Prompt injection (LLM-1, LLM-8)", "RAG poisoning via KB (LLM-2, T-6)", "LLM output contains shell metacharacters or SQL fragments", and 5 additional actions.

Remediation Steps

- Implement strict tool call parameter validation at the MCP Tool Server: define a JSON Schema for each registered tool's `params` object and validate all incoming parameters against it before dispatch.
- Reject calls where parameters fail schema validation.
- Implement parameter sanitization stripping SQL injection, shell metacharacters, and path traversal sequences from string parameters.

Cross-Layer Attack Chain Analysis

Cross-Layer Attack Chain Analysis

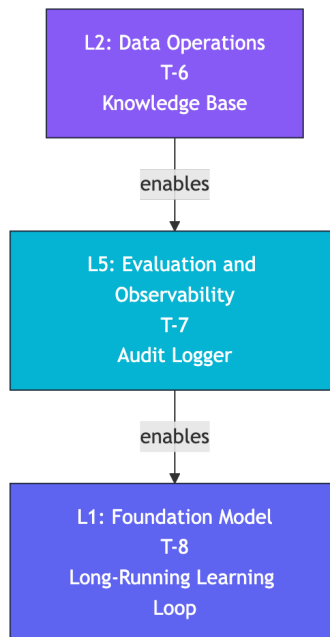
CRITICAL CHAIN-001 L7 → L6 → L3 **User Identity Spoofing to Agent Framework Privilege Escalation via Security Controls**


Attack Progression

An attacker initiates the chain by replaying stolen session tokens or forging identity credentials at the User→Guardrails boundary (S-1 at L7), impersonating a legitimate user to inject a crafted prompt into the agentic pipeline. The impersonated user identity shifts to the Security and Compliance layer, where the adversarial prompt bypasses Guardrails filtering controls (E-1 at L6). The prompt injection that circumvents the Guardrails Service effectively elevates the attacker from "unauthenticated external entity" to "trusted caller of the LLM Agent Orchestrator" — the attacker's prompt carries the same trust level as legitimately validated inputs. This security-layer bypass enables the attacker to issue commands to the Orchestrator that appear to originate from an authenticated session. The privilege escalation at the security layer triggers escalation at the Agent Framework layer (E-5 at L3). With the Orchestrator now processing adversarially-crafted inputs as trusted, it issues tool call requests to the MCP Tool Server under the attacker's injected instructions. Without zero-trust authorization at the Tool Server, the attacker gains the Tool Server's full execution privileges — service account credentials, API keys, and external write access — manifesting as complete agent framework compromise. Remediating S-2 (the Guardrails Service spoofing finding, chain-breaking target at L6) would require mTLS between Guardrails and Orchestrator, preventing bypass of the security layer and breaking the chain at its structural midpoint.

Impacted Findings: S-1, E-1, E-5

Cross-Layer Attack Chain Analysis

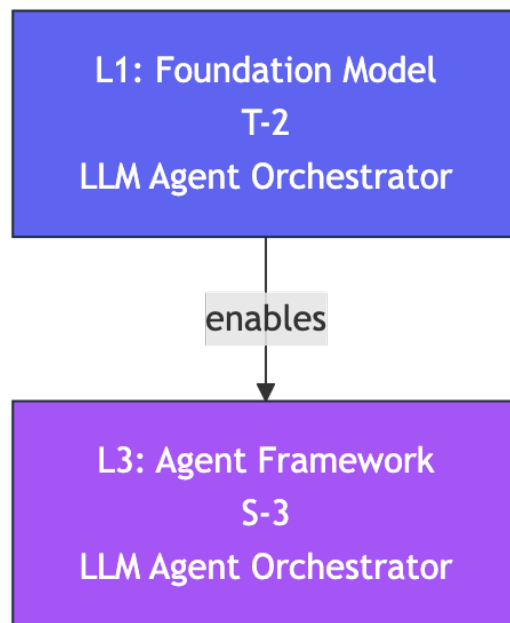
CRITICAL CHAIN-002 L2 → L5 → L1 **Data Poisoning to Temporal Model
Compromise via Observability Gap**


Attack Progression

The chain begins at the Data Operations layer (T-6 at L2) where an attacker with write access to the Knowledge Base injects adversarial documents into the vector store. Beyond corrupting Orchestrator context retrieval, these adversarial documents — once retrieved and logged as part of agent decision records — flow into the Audit Logger as operational history. The data operations tampering triggers observability layer corruption (T-7 at L5). The Audit Logger collects decision logs from all Application Zone components including knowledge-base-retrieval records. If the adversarial KB content appears in the Orchestrator's decision logs, it enters the Audit Logger alongside legitimate operational data. An attacker can additionally directly tamper with Audit Logger entries, destroying the ability to detect the KB poisoning and simultaneously injecting fabricated training signals. The corrupted audit log stream then shifts the attack surface to the model layer. The compromised observability layer manifests as temporal model compromise at the Foundation Model layer (T-8). The Learning Loop consumes the corrupted Audit Logger training signal stream and incorporates adversarially-crafted patterns into model updates distributed to the Orchestrator, Specialist, and Clinical Advisory Sub-Agent. The time-delayed activation — the model update cycle introduces latency between poison injection and behavioral impact — makes this chain difficult to detect until the updated model begins exhibiting attacker-preferred behaviors. Remediating T-7 (the Audit Logger tampering finding at L5) is the chain-breaking point: an append-only, Merkle-hash-verified audit store prevents the observability layer from acting as a corruption relay between the data layer and the model layer.

Impacted Findings: T-6, T-7, T-8

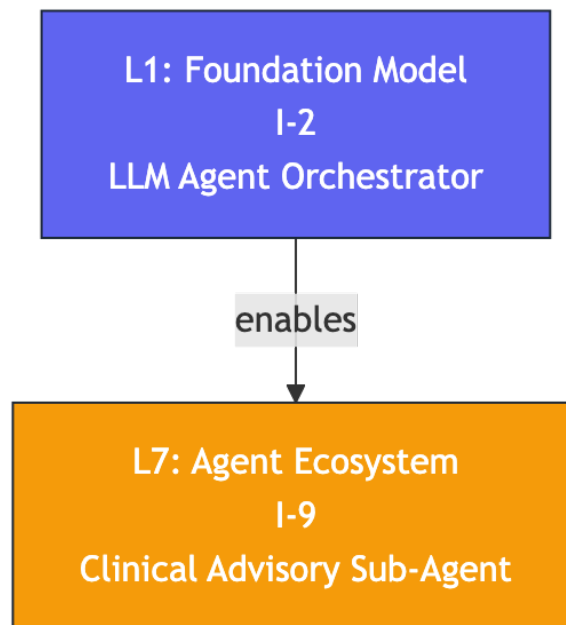
Cross-Layer Attack Chain Analysis

CRITICAL CHAIN-003 L1 → L3 **Foundation Model Tampering to Agent Spoofing via Corrupted Context****Attack Progression**

The chain initiates at the Foundation Model layer (T-2 at L1) where an attacker tampers with the LLM Agent Orchestrator's context window by corrupting any upstream data source — the Knowledge Base (via poisoned vector search results), the Inter-Agent Channel (via injected aggregated results), or tool call responses from the MCP Tool Server. Adversarial content injected into the context manipulates the Orchestrator's reasoning process at the foundation model level. The manipulated model responses at L1 enable spoofing at the Agent Framework layer (S-3 at L3). When the Orchestrator's context has been corrupted, it may generate delegation messages to the Specialist Agent that appear to originate from its own authorized reasoning but actually implement attacker-directed instructions. Since the Orchestrator's identity is not cryptographically attested in delegation messages, a compromised Orchestrator becomes indistinguishable from a rogue process injecting unauthorized instructions via the Inter-Agent Channel — the tampering at the model layer manifests as agent identity spoofing at the framework layer. Remediating T-2 (the chain's sole and initial finding at L1) is the structural chain-breaking target: validating the integrity of all context sources before injection into the Orchestrator's context window prevents the model-layer corruption that enables downstream agent spoofing.

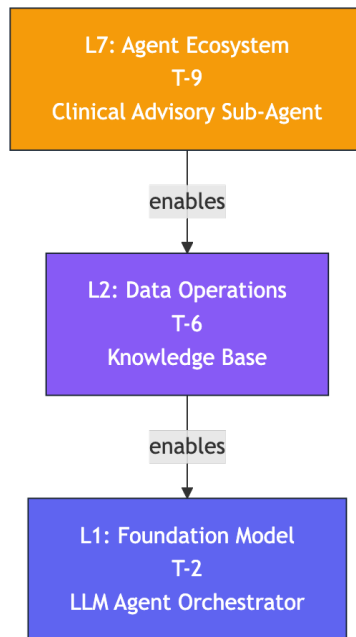
Impacted Findings: T-2, S-3

Cross-Layer Attack Chain Analysis

CRITICAL CHAIN-004 L1 → L7 **Foundation Model Information Disclosure to Ecosystem Leakage****Attack Progression**

The chain initiates at the Foundation Model layer (I-2 at L1) where a prompt injection attack or model hallucination causes the LLM Agent Orchestrator to leak its context window content — retrieved Knowledge Base documents, tool results, system prompts — in its outputs. This information disclosure at the model layer includes sensitive clinical context that the Orchestrator receives from the Clinical Advisory Sub-Agent's clinical summaries. The model-layer leakage manifests as ecosystem-level information disclosure (I-9 at L7). The Clinical Advisory Sub-Agent returns Clinical Summary + Recommendations to the Orchestrator, which may include patient-specific clinical data, EHR document excerpts, and proprietary clinical protocols. When the Orchestrator's context window leaks, this clinical information — now embedded in the Orchestrator's reasoning context — surfaces in the HTTPS response to the User, exposing sensitive clinical data to unauthorized parties. Additionally, Clinical Decision Log Entries flowing to the Audit Logger carry this clinical information into the training stream without field-level classification, creating a secondary leakage path at the ecosystem boundary. Remediating I-2 (the Foundation Model layer disclosure at L1) is the chain-breaking point: implementing output scrubbing on the Orchestrator's response before HTTPS transmission prevents model-layer leakage from propagating to the ecosystem boundary.

Cross-Layer Attack Chain Analysis

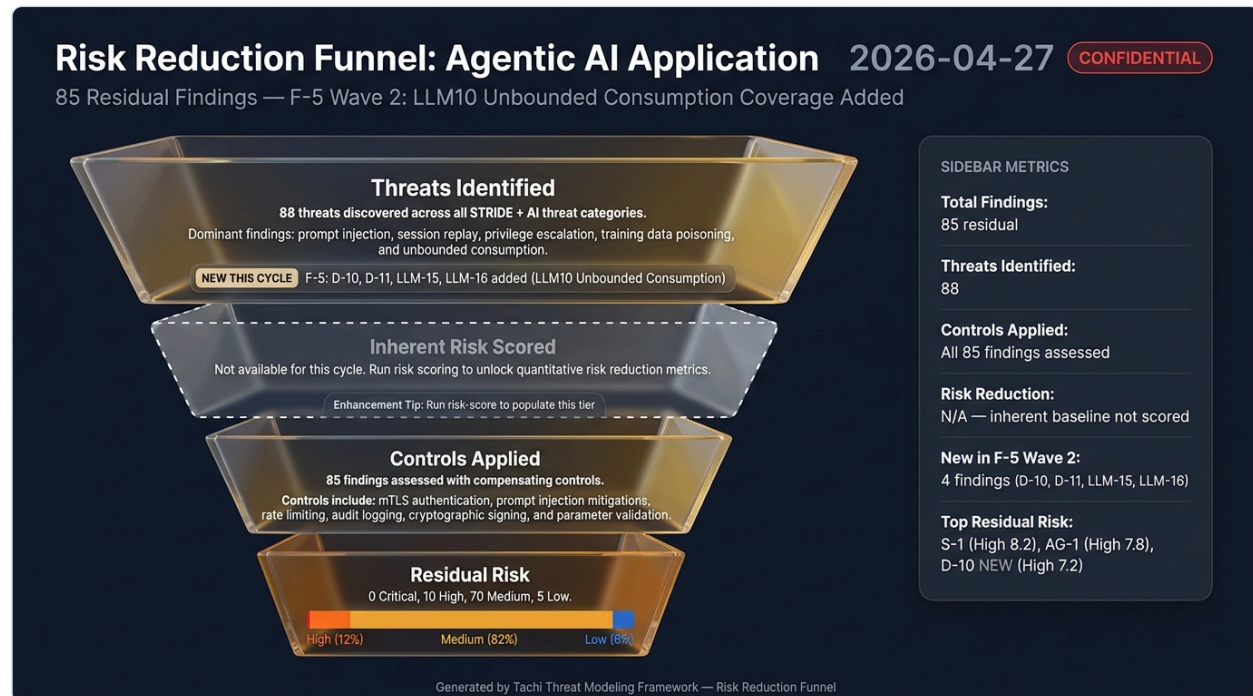
CRITICAL CHAIN-005 L7 → L2 → L1 **Clinical Sub-Agent Data Operations
Tampering to Foundation Model Corruption**


Attack Progression

The chain begins at the Agent Ecosystem layer with the Clinical Advisory Sub-Agent (T-9 at L7). An attacker compromises the Clinical Advisory Sub-Agent's context assembly by injecting adversarial content into the Clinical Query / Context payload from the Orchestrator, or by targeting the Knowledge Base writes that the sub-agent reads during vector search. The sub-agent, once compromised at the ecosystem layer, triggers corruption of the shared Knowledge Base. The ecosystem-layer tampering triggers data operations layer corruption (T-6 at L2). The Clinical Advisory Sub-Agent writes its retrieved-document interactions back as log entries and its summarization patterns influence the Audit Logger's training signal stream. More directly, an attacker who compromises the sub-agent's context (via the clinical query payload) can cause the sub-agent to issue vector search queries designed to test and identify document boundaries, enabling the attacker to target specific Knowledge Base document writes. The corrupted KB then affects Orchestrator retrievals. The poisoned Knowledge Base manifests as Foundation Model layer corruption (T-2 at L1). The LLM Agent Orchestrator relies on the same Knowledge Base for context retrieval. Adversarial documents that were injected to affect the Clinical Advisory Sub-Agent's outputs are also retrieved by the Orchestrator during its vector searches, injecting the same adversarial content into the Orchestrator's context window, corrupting its reasoning, and enabling all downstream attack chains that originate from T-2. Remediating T-6 (the Knowledge Base tampering at L2) is the structural midpoint and chain-breaking target: implementing write access controls and document-level integrity verification on the Knowledge Base prevents the data operations layer from acting as the conduit between the ecosystem and foundation model layers.

Impacted Findings: T-9, T-6, T-2

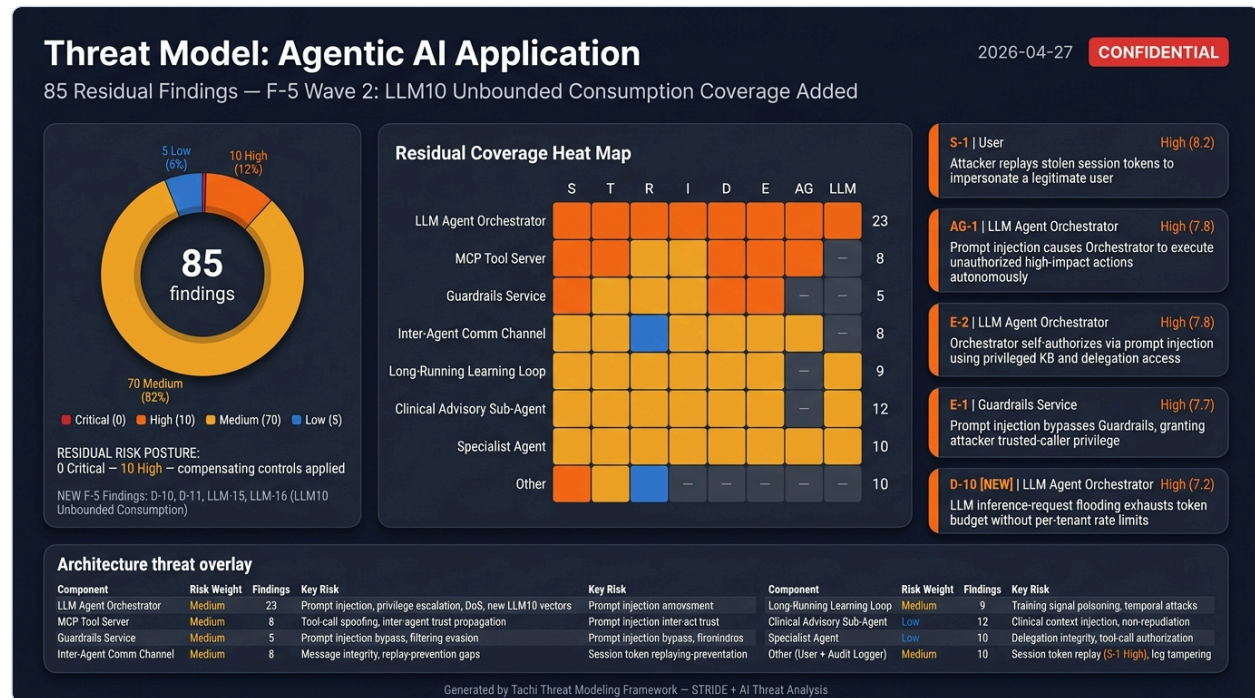
Risk Reduction Funnel



The Risk Reduction Funnel illustrates how the overall risk profile transforms through each stage of the assessment pipeline. The top tier shows all threats identified during the initial STRIDE and AI-specific threat analysis. The second tier reflects severity recalibration after quantitative scoring, where composite scores incorporating CVSS, exploitability, scalability, and reachability may shift initial severity ratings.

The third tier shows the impact of compensating controls detected in the codebase — existing mitigations that reduce effective risk. The bottom tier presents the final residual risk posture, representing the organization's actual exposure after accounting for all identified defenses. The sidebar provides key metrics at a glance, including the total risk reduction percentage, control coverage rate, and the highest-residual-risk finding requiring priority attention.

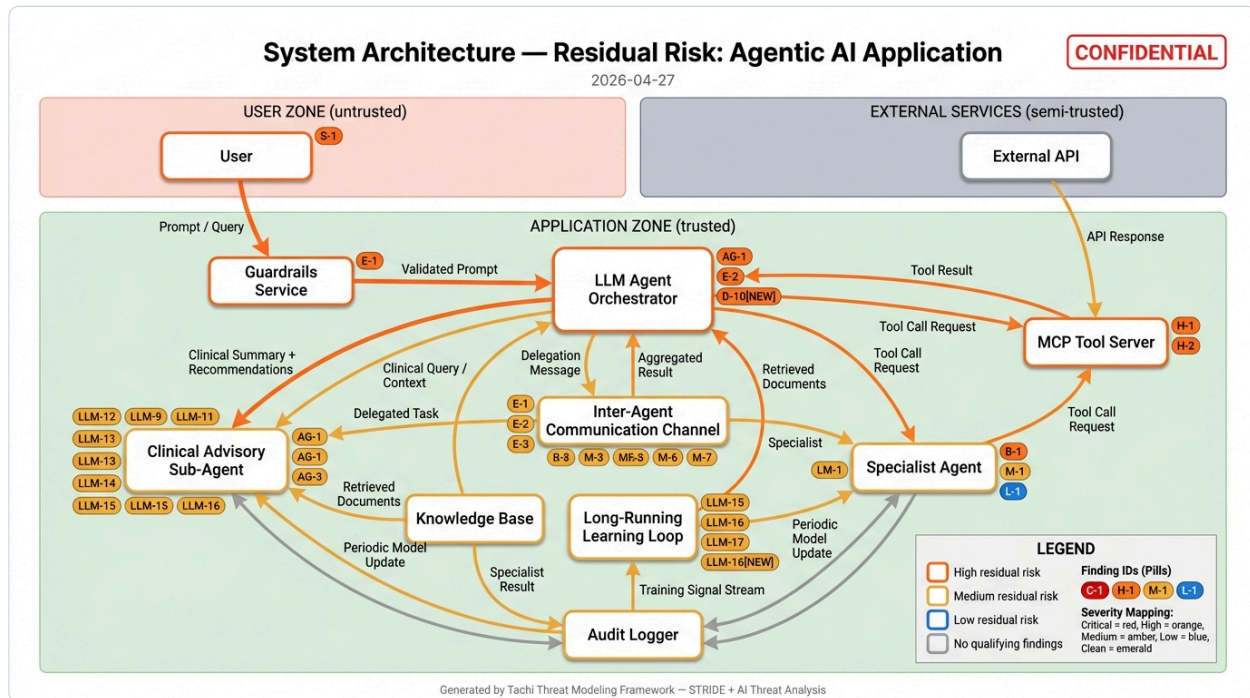
Risk Summary Dashboard



The Risk Summary Dashboard consolidates assessment results into three complementary views. The left panel displays the residual risk distribution across severity levels, showing how findings are distributed after compensating controls are applied.

The center panel presents a component-by-threat-category heat map, where each cell indicates the highest residual severity for that intersection of system component and STRIDE or AI threat category. This enables rapid identification of which components carry the highest concentration of risk. The right panel highlights the top findings by residual risk score, listing the specific threats that require the most immediate attention with their affected components and residual scores.

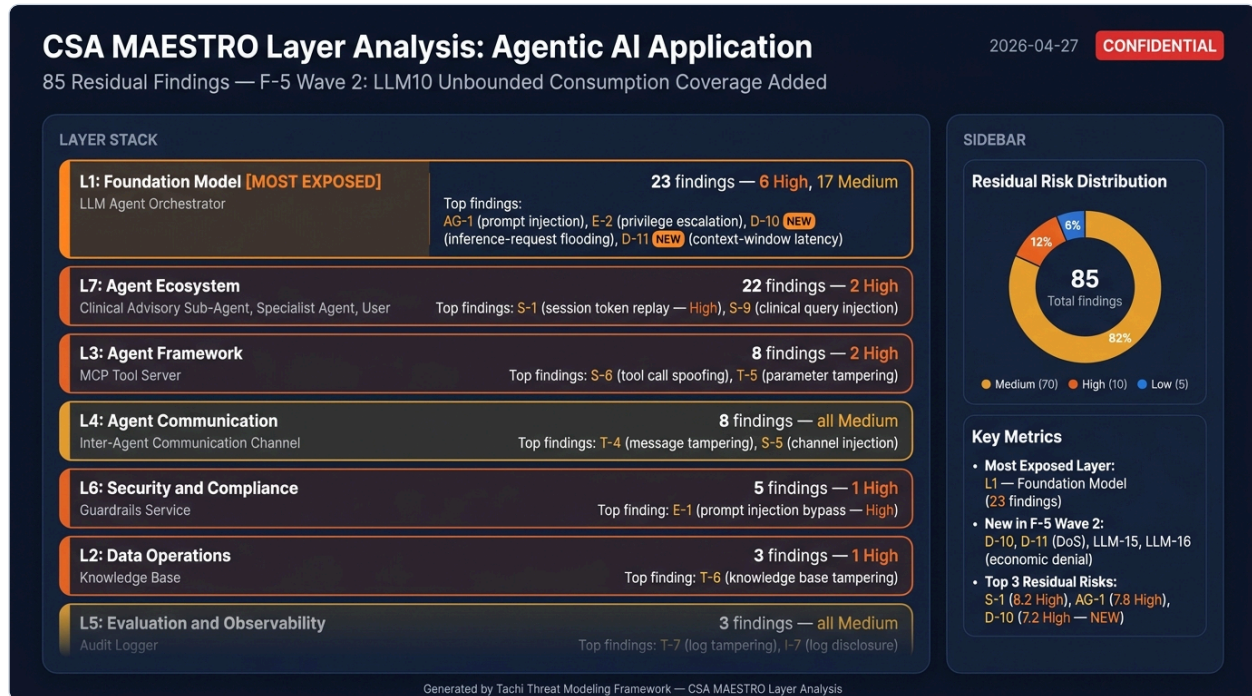
System Architecture



The System Architecture diagram maps identified threats directly onto the assessed system's component topology. Components are grouped within their trust boundaries — external/untrusted, application layer, and infrastructure/platform zones — with color coding reflecting the concentration and severity of findings at each location.

Threat badges on each component show the specific threat identifiers and their severity classifications. Data flow arrows illustrate the assessed communication paths between components, with trust boundary crossings highlighted as key attack surface areas where threats are most likely to be exploited. This spatial view complements the tabular findings detail by showing how threats cluster around specific architectural chokepoints.

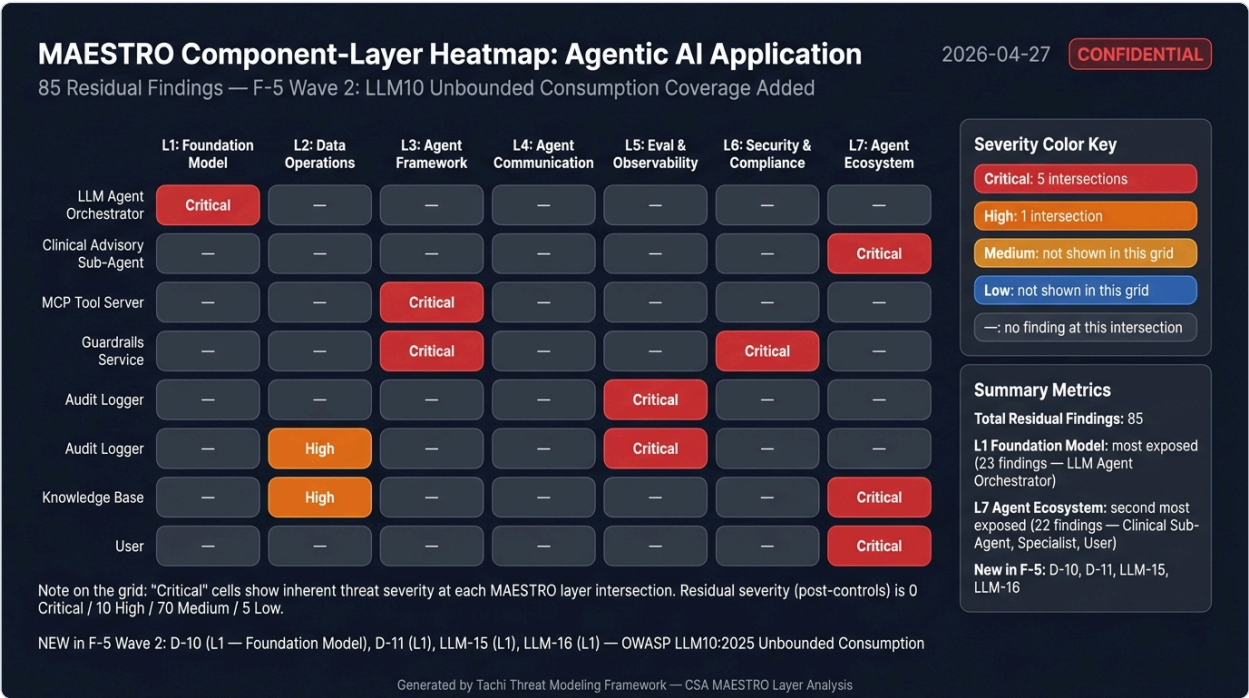
MAESTRO Layer Risk Distribution



The MAESTRO Layer Risk Distribution diagram maps identified threats onto the seven-layer CSA MAESTRO framework for agentic AI systems. Each horizontal band represents an architectural layer from Foundation Model (L1) through Agent Ecosystem (L7), showing the finding count and highest severity at each layer. The most-exposed layer is visually highlighted.

This view enables security leaders to quickly identify which architectural layers of the AI stack carry the most risk exposure and prioritize remediation efforts accordingly.

MAESTRO Component-Layer Heatmap



The MAESTRO Component-Layer Heatmap provides a granular view of how threats intersect specific system components with MAESTRO architectural layers. Each cell represents the highest-severity finding at the intersection of a component (row) and a MAESTRO layer (column).

This cross-tabulation identifies specific component-layer hotspots that require targeted remediation, complementing the aggregate layer view in the MAESTRO Stack diagram.

MAESTRO Layer Analysis

The following findings are grouped by their CSA MAESTRO framework architectural layer, providing a layer-oriented view of the threat landscape. Each layer represents a distinct level of the agentic AI system stack, from Foundation Model (L1) through Agent Ecosystem (L7). Findings within each layer are sorted by severity.

L1 — Foundation Model (22 findings)

ID	Component	Severity	Threat
S-3	LLM Agent Orchestrator	CRITICAL	The Orchestrator's identity is not cryptographically attested to the Specialist Agent via the Inter-Agent Communication Channel. A compromised or rogue process could inject messages into the channel impersonating the Orchestrator, issuing unauthorized delegation instructions to the Specialist Agent.
T-2	LLM Agent Orchestrator	CRITICAL	The Orchestrator's context window (system prompt, retrieved documents, tool results) can be tampered with by an attacker who controls any upstream data source — the Knowledge Base, the Inter-Agent Channel (aggregated results), or tool call responses from the MCP Tool Server. Injecting adversarial content into the context manipulates the Orchestrator's reasoning and outputs.
R-3	LLM Agent Orchestrator	CRITICAL	The Orchestrator denies having issued a specific delegation message or tool call request, claiming the action was performed by another process. Without per-action logging of Orchestrator-originated actions with content hashes, the Orchestrator's operational history cannot be audited.
I-2	LLM Agent Orchestrator	CRITICAL	The Orchestrator's context window contains sensitive information (retrieved documents from the Knowledge Base, tool results, system prompts). A prompt injection attack or model hallucination can cause the Orchestrator to leak this context in its HTTPS response to the User, exposing system-internal data.
D-2	LLM Agent Orchestrator	CRITICAL	The Orchestrator's inference pipeline is a bounded resource. An attacker (or malfunctioning upstream component) can exhaust the Orchestrator's capacity by flooding it with high-token-count prompts or by injecting context that forces recursive tool invocation chains. This starves legitimate user requests of Orchestrator capacity.
D-10	LLM Agent Orchestrator	CRITICAL	LLM Inference-Request Flooding and Token Exhaustion (Pattern Category 12 — OWASP LLM10:2025): The LLM Agent Orchestrator exposes an LLM inference pipeline without declared per-tenant queries-per-second (QPS) rate limiting at the API-gateway layer that is distinct from generic per-IP network-layer rate limiting. An attacker with valid authenticated access (or a compromised credential) can flood the Orchestrator's inference endpoint with concurrent

			requests each carrying maximal prompt-token payloads, exhausting per-request inference compute and starving legitimate users of Orchestrator capacity. Token-counting middleware is not declared — the system accepts requests without checking projected per-call cost, learning total token usage only after-the-fact via billing reconciliation. Per-tenant token budget per request is not declared at the inference gateway — a single attacker session can request the model's maximum output (up to full context-window completion tokens), creating per-call cost asymmetry between attacker and operator. Request timeouts configured for typical non-LLM request latency may prematurely kill legitimate long-inference calls while the attack's short-but-high-volume requests occupy the queue. The Orchestrator also fans out to the Specialist Agent and Clinical Advisory Sub-Agent, multiplying the inference cost per user request — a single attacker request can trigger three concurrent LLM inference calls, amplifying the denial-of-service surface beyond per-endpoint rate-limiting controls.
D-11	LLM Agent Orchestrator	CRITICAL	Context-Window Exhaustion — Latency-Driven Variant (Pattern Category 13 — OWASP LLM10:2025; Q1 SPLIT Vector A): The LLM Agent Orchestrator accepts conversation history and contextual payloads via the Guardrails→Orchestrator data flow without a declared max-context-window enforcement policy at the API gateway. An attacker constructs an adversarially long prompt — embedding a full 32k+ token conversation history or injecting recursive prompt-expansion templates that cause the Orchestrator to re-engage its own outputs (chain-of-thought recursion, self-reflection loops, multi-step reasoning expansion) — driving per-request context-window usage to 99% of model maximum. This spikes per-request inference latency from a typical 1–2 seconds to the per-tenant timeout threshold (often 30–60 seconds), causing legitimate requests to queue behind the blocked inference slot. The Orchestrator's fan-out to Specialist Agent and Clinical Advisory Sub-Agent compounds this: a single max-context request causes three concurrent long-latency inference calls, effectively blocking three inference slots simultaneously. Context-window monitoring is not declared — no anomaly alerting on percentage-of-max usage spikes per request or per-tenant — making the attack invisible until latency degradation is user-visible. Per-tenant context-window cap is not differentiated from per-request cap, allowing consecutive max-context requests from a single tenant to monopolize the inference cluster's per-slot latency budget. Note: Vector A (latency-driven availability disruption) is this finding. Vector B (cost-driven denial-of-wallet) is covered by LLM-16 in the model-theft / LLM section per Q1 SPLIT ADR-034 Decision 3.
E-2	LLM Agent Orchestrator	CRITICAL	The Orchestrator has privileged access to the Knowledge Base, MCP Tool Server, and delegation authority over the Specialist Agent and Clinical Advisory Sub-Agent. A prompt injection attack that manipulates the Orchestrator's reasoning can cause it to self-authorize elevated operations: exfiltrating the full KB corpus, invoking tools outside the user's permitted scope, or issuing unauthorized delegation messages.
AG-1	LLM Agent Orchestrator	CRITICAL	Prompt injection causes the Orchestrator to autonomously execute unauthorized high-impact actions (mass data exfiltration from KB, bulk tool invocations against External API) beyond the scope of the user's original request,

			exploiting the Orchestrator's broad access to system capabilities.
AG-2	LLM Agent Orchestrator	CRITICAL	The Orchestrator and Specialist Agent can jointly coordinate (via the Inter-Agent Channel) to achieve a combined action that neither could perform alone or that would trigger per-agent rate limits if attempted individually. An attacker who compromises both agents (or injects coordinated prompts via the Inter-Agent Channel) can leverage this coordination for policy circumvention or joint data exfiltration.
LLM-1	LLM Agent Orchestrator	CRITICAL	Direct prompt injection via the User→Guardrails→Orchestrator chain: an attacker embeds adversarial instructions in the user's prompt that the Guardrails Service fails to detect, causing the Orchestrator to override its system prompt, reveal internal configuration, or execute unauthorized actions.
LLM-2	LLM Agent Orchestrator	CRITICAL	Indirect prompt injection via the Knowledge Base: an attacker embeds adversarial instructions in documents stored in the Knowledge Base. When the Orchestrator retrieves these documents during vector search, the adversarial instructions are injected into its context window, hijacking its reasoning.
LLM-4	LLM Agent Orchestrator	CRITICAL	Training data poisoning via the Long-Running Learning Loop: the Orchestrator ingests model updates from the Learning Loop that was trained on audit log data. An attacker who pollutes the audit log with adversarial interaction records (fabricated user sessions designed to shift model behavior) poisons the Orchestrator's future behavior at update time.
LLM-5	LLM Agent Orchestrator	CRITICAL	Improper output handling — client-side XSS via the Orchestrator's HTTPS response to the User: LLM-generated response content rendered as HTML in the user's browser without contextual output encoding enables stored or reflected XSS. An attacker who primes the Orchestrator (via prompt injection or RAG poisoning) to emit ` <script>` or event-handler payloads causes client-side execution under the application's origin with access to session cookies and CSRF tokens. Execution context is client-side: the payload runs in the victim user's browser.</td></tr> <tr> <td>LLM-6</td><td>LLM Agent Orchestrator</td><td>CRITICAL</td><td>Improper output handling — server-side execution via Tool Call Request: LLM-generated JSON-RPC parameters flowing into the MCP Tool Server may contain injection payloads (SQL fragments, shell metacharacters) that are executed server-side when the Tool Server dispatches tool invocations. Execution context is server-side: the payload runs on the Tool Server with its service account credentials and external API access.</td></tr> <tr> <td>LLM-15</td><td>LLM Agent Orchestrator</td><td>CRITICAL</td><td>Cost Amplification via Recursive or Cost-Asymmetric Prompting (Pattern Category 10 — OWASP LLM10:2025): The LLM Agent Orchestrator accepts prompts from the Guardrails Service without declared recursive-prompt depth limits or output-token caps tuned to realistic response-length p99. The Orchestrator fans out to the Specialist Agent (via Inter-Agent Channel) and to the Clinical Advisory Sub-Agent (via JSON-RPC), both of which may themselves invoke the Knowledge Base (via RAG fan-out) and the MCP Tool Server (via tool calls). This multi-hop agent loop — Orchestrator → Specialist → ToolServer → ExtAPI, and Orchestrator → ClinAdvisor → KB — creates a recursive cost-amplification surface: an adversarial 10-token user prompt can trigger a chain-of-thought response from the Orchestrator, delegation to the Specialist and ClinAdvisor,</td></tr> </table> </div> <div data-bbox="471 966 523 982" data-label="Page-Footer">Page 40</div> <div data-bbox="764 966 889 982" data-label="Page-Footer">Generated by tachi</div></script>

			RAG fan-out from KB, and tool calls to ToolServer and ExtAPI, generating 32k+ tokens of combined output across all three inference endpoints. Output-amplification ratio (output-tokens / input-tokens) is not declared as a monitored metric — pathological ratios above 100x from recursive chain-of-thought expansion are invisible without per-request amplification telemetry. Cost-per-query p99 alerting is not declared — sustained cost-amplification attacks register as gradual cost increases that per-request rate limits do not catch because each individual request may be under the per-request token cap while the aggregate exceeds the operator's revenue-per-query threshold. Per-tenant cost-amplification anomaly detection is not declared — without per-tenant cost-velocity monitoring the operator cannot identify which session is driving amplification. Note: MITRE ATT&CK T1496 (Resource Hijacking) is a relevant cross-reference for the attacker's goal of consuming the operator's inference compute at the operator's expense; T1496 is cited here as prose context only (not in references array — T1496 is not catalog-resolvable in schemas/taxonomy/mitre-attack.yaml per ADR-034 Decision 6).
0I-1	LLM Agent Orchestrator	CRITICAL	<p>Improper output handling — client-side XSS via LLM response rendered in user browser: the Orchestrator's "Response (HTTPS)" data flow sends LLM-generated content directly to the User. If the client-side rendering layer injects this content into the DOM via `innerHTML` or equivalent without HTML entity encoding, an attacker who primes the Orchestrator to emit `<script >`="" (1)="" (2)="" (browser="" (https)`="" `response="" a="" access="" agent="" an="" and="" apis.="" application's="" browser="" causes="" client-side="" confirmed:="" cookies,="" csrf="" data="" direct="" downstream="" event-handler="" execution="" flow="" fr-011="" gate="" in="" into="" is="" keyword="" llm="" on="" or="" orchestrator,="" origin,="" output-sink="" p="" path:="" payload="" process="" render="" session="" sink).<="" src="//evil.example/steal.js" surface="client-side" surface.="" the="" this="" to="" tokens,="" two-part="" under="" user="" user-authenticated="" victim's="" with="" →=""> </td></tr> <tr> <td>0I-2</td><td>LLM Agent Orchestrator</td><td>CRITICAL</td><td> <p>Improper output handling — server-side execution via Tool Call Request: the Orchestrator emits "Tool Call Request (JSON-RPC)" messages to the MCP Tool Server containing LLM-synthesized parameters. If tool parameters are used to construct SQL queries, shell commands, template expressions, or filesystem paths server-side without parameterization or sanitization, an attacker who influences the Orchestrator's output achieves server-side code/command execution via the tool execution sink. Execution context: Tool Server backend, with service account credentials and External API access. FR-011 gate confirmed: (1) LLM keyword on Orchestrator, (2) Tool Call Request (JSON-RPC) → MCP Tool Server = server-side execution sink.</p> </td></tr> <tr> <td>LLM-3</td><td>LLM Agent Orchestrator</td><td>HIGH</td><td> <p>Model theft via systematic API probing: an attacker issues carefully crafted queries to extract the Orchestrator's model behavior, fine-tuning data characteristics, or system prompt contents through systematic probing, enabling the attacker to build a functional replica of the model or extract proprietary training data.</p> </td></tr> <tr> <td>LLM-7</td><td>LLM Agent Orchestrator</td><td>HIGH</td><td> <p>Improper output handling — SSRF via LLM-synthesized URL: the Orchestrator can instruct the MCP Tool Server to fetch external URLs synthesized from LLM output. An attacker who influences the Orchestrator's output can cause it to emit internal service URLs (cloud metadata endpoints,</p> </td></tr> </table> </div> <div data-bbox="471 967 523 982" data-label="Page-Footer">Page 41</div> <div data-bbox="764 967 889 982" data-label="Page-Footer">Generated by tachi</div></script></p>

			internal admin APIs) as tool parameters. The Tool Server fetches these with its server-side network credentials. Execution context is server-side: the HTTP client runs with the Tool Server's IAM role.
LLM-16	LLM Agent Orchestrator	HIGH	Denial-of-Wallet via Context-Window Cost Amplification (Pattern Category 11 — OWASP LLM10:2025; Q1 SPLIT Vector B): The LLM Agent Orchestrator's context-window handling lacks per-tenant token budget hard-cap at the API gateway and at-query-time cost reconciliation. An attacker can drive the Orchestrator's context-window to model maximum on each call (Vector B — cost-driven economic damage, distinct from D-11's Vector A latency-driven availability disruption) by submitting large conversation histories or recursive prompts that inflate context usage. The Orchestrator's fan-out to Specialist Agent and Clinical Advisory Sub-Agent compounds per-call cost: a single max-context Orchestrator request triggers additional max-context calls to Specialist and ClinAdvisor, multiplying the per-request inference bill by up to 3x. Cost-per-query p99 alerting is not declared — denial-of-wallet attacks accumulate as gradual billing increases invisible to per-request rate limits. Denial-of-wallet anomaly detection (cost-velocity monitoring across 5-minute, 1-hour, 24-hour windows) is not declared — sustained billing-amplification attacks are invisible until the operator's invoice arrives. Automated tenant suspension on budget breach is not declared — by the time a human reviews a cost-velocity anomaly, the operator's bill is already inflated. Per-tenant billing attribution is not declared at-query-time — without synchronous attribution the operator cannot enforce per-tenant token budgets pre-inference. Severity Q3 RESOLVED evaluation: HIGH default applies (no multi-tenant freemium structure structurally evident in architecture; single-application topology; CRITICAL 2-condition floor not met — condition (a) B2C freemium multi-tenant NOT confirmed, condition (b) assessment deferred). Note: MITRE ATT&CK T1496 (Resource Hijacking) is a relevant cross-reference for the attacker's economic-damage goal; T1496 cited as prose context only (NOT in references array — not catalog-resolvable per ADR-034 Decision 6).
OI-3	LLM Agent Orchestrator	HIGH	Improper output handling — SSRF via LLM-synthesized URL in Tool Call Request: the Orchestrator constructs "Tool Call Request (JSON-RPC)" messages containing URLs sourced from LLM output (e.g., "fetch this URL" tool calls). The MCP Tool Server executes outbound HTTP to the supplied URL using its own server-side network credentials and IAM role. An attacker who influences the Orchestrator's output can cause it to emit `http://169.254.169.254/latest/meta-data/iam/security-credentials/` or RFC 1918 internal service URLs. Execution context is server-side: the HTTP client runs with the Tool Server's IAM role and internal network reach. FR-011 gate confirmed: (1) LLM keyword on Orchestrator, (2) Tool Call Request (JSON-RPC) → MCP Tool Server → External API (HTTP fetch) = SSRF-capable server-side execution sink.

L2 — Data Operations (3 findings)

ID	Component	Severity	Threat
----	-----------	----------	--------

T-6	Knowledge Base	HIGH	The Knowledge Base corpus can be tampered with (poisoned) by an attacker who gains write access. Injecting adversarial documents into the knowledge store causes the Orchestrator to retrieve and incorporate malicious context during vector search, corrupting the Orchestrator's responses at scale.
I-6	Knowledge Base	HIGH	The Knowledge Base exposes its full document corpus to any process that can issue a vector search query. Without query-result access controls, a compromised Orchestrator or injected context can exfiltrate the entire corpus by issuing exhaustive search queries.
D-6	Knowledge Base	MEDIUM	The Knowledge Base can be rendered unavailable by an attacker who issues high-volume, complex vector search queries (exhaustive nearest-neighbor searches with high dimensionality). This degrades retrieval performance for the Orchestrator.

L3 — Agent Framework (8 findings)

ID	Component	Severity	Threat
S-6	MCP Tool Server	CRITICAL	An attacker in the Application Zone spoofs a valid agent (Orchestrator or Specialist) to submit unauthorized tool call requests to the MCP Tool Server. Without caller authentication, any process can invoke tools with the server's external-facing credentials.
T-5	MCP Tool Server	CRITICAL	Tool call request parameters supplied by agent LLM outputs can be tampered with before execution if the Tool Server does not validate them against an explicit allowlist. An attacker who can influence the Orchestrator's or Specialist's LLM output can modify JSON-RPC parameters to call unintended tools or supply malicious arguments (e.g., injecting shell metacharacters into tool parameters).
D-5	MCP Tool Server	CRITICAL	The Tool Server's capacity for concurrent External API calls is bounded by rate limits imposed by the API provider and by the Tool Server's own connection pool. A compromised agent sending high-volume tool call requests can exhaust the connection pool, causing all legitimate tool calls to fail.
E-5	MCP Tool Server	CRITICAL	The MCP Tool Server executes tools with credentials it holds (service account tokens, API keys for External API). If an agent sends unauthorized tool calls (via forged identity or exploited Orchestrator), it gains the Tool Server's execution privileges — invoking external APIs, writing to external systems, and accessing data sources with the server's full credential set.
AG-5	MCP Tool Server	CRITICAL	The MCP Tool Server is vulnerable to tool call injection: an attacker who can influence the LLM output of either the Orchestrator or Specialist Agent can inject crafted JSON-RPC parameters that invoke unintended tools (tool name injection) or supply malicious arguments to permitted tools (parameter injection). The Tool Server executes these with its own service credentials.
R-6	MCP Tool Server	HIGH	The MCP Tool Server denies having executed a specific tool invocation or received a particular JSON-RPC request. Without signed execution logs, tool invocations cannot be attributed to the requesting agent.
I-5	MCP Tool Server	HIGH	Tool results from External API calls may contain sensitive data (user records, financial data, PII) that the Tool Server logs verbatim to the Audit Logger. If Audit Logger access is

			not restricted, this data becomes accessible to any process that reads the audit trail, including the Learning Loop's training pipeline.
AG-6	MCP Tool Server	HIGH	The MCP Tool Server acts as a privileged execution broker. Runaway or adversarially prompted agents (Orchestrator or Specialist) can cause the Tool Server to repeatedly call External API endpoints in rapid succession, exhausting the API provider's rate limits, incurring financial costs, or triggering security lockouts that deny the system access to required external capabilities.

L4 — Deployment Infrastructure (0 findings)

Analyzed — no findings this scan.

L5 — Evaluation and Observability (3 findings)

ID	Component	Severity	Threat
I-7	Audit Logger	CRITICAL	The Audit Logger aggregates sensitive data from all Application Zone components. Unauthorized read access to the logger (misconfigured access controls, insider threat) exposes the full operational history of the agent system, including user prompts, model decisions, tool call parameters, and filter rule triggers.
T-7	Audit Logger	HIGH	The Audit Logger entries can be tampered with by a process with write access to the log store. Modifying or deleting log entries corrupts the training signal stream consumed by the Long-Running Learning Loop, causing poisoned model updates, and also destroys forensic evidence needed for incident response.
D-7	Audit Logger	HIGH	The Audit Logger can be overwhelmed by a log-flooding attack from a compromised Application Zone process, causing legitimate log entries to be dropped or the logger to become unavailable, creating audit gaps and potentially cascading to block all pipeline operations that wait for log confirmation.

L6 — Security and Compliance (6 findings)

ID	Component	Severity	Threat
D-1	Guardrails Service	CRITICAL	The Guardrails Service is vulnerable to resource exhaustion via high-volume prompt submission. An attacker sends computationally expensive prompts (complex regex evaluation patterns, adversarially crafted inputs that maximize rule evaluation cost) at high rate to degrade or collapse the filtering pipeline.
E-1	Guardrails Service	CRITICAL	A prompt injection attack that bypasses the Guardrails Service effectively elevates the attacker's privilege from "unauthenticated user" to "trusted caller of the Orchestrator". The attacker's prompt reaches the Orchestrator with the same trust level as validated internal inputs, enabling subsequent escalation.
S-2	Guardrails Service	HIGH	An attacker spoofs the Guardrails Service by sending crafted requests directly to the LLM Agent Orchestrator's internal

			endpoint, bypassing validation entirely. If internal service endpoints lack mutual TLS authentication, any service within the Application Zone can impersonate the Guardrails.
T-1	Guardrails Service	HIGH	An attacker with write access to the Guardrails Service configuration (via a misconfigured admin endpoint or insider threat) modifies filtering rules to allow previously-blocked prompt patterns through to the Orchestrator, silently bypassing content policy enforcement.
R-2	Guardrails Service	MEDIUM	The Guardrails Service can deny having logged a filtering event or claim that an input passed filtering when it was actually rejected (or vice versa). Without tamper-evident logs, the filtering pipeline's decisions cannot be verified independently.
I-1	Guardrails Service	MEDIUM	The Guardrails Service leaks the content of rejected prompts (including their rejection reasons) in error responses returned to the User. An attacker can iteratively probe the filtering rules by observing rejection reasons, enabling systematic bypass through adaptive prompt crafting.

L7 — Agent Ecosystem (14 findings)

ID	Component	Severity	Threat
S-1	User	CRITICAL	An attacker impersonates a legitimate user by replaying stolen session tokens or forging identity credentials to bypass authentication at the User→Guardrails boundary, gaining unauthorized access to the system under a victim identity.
S-9	Clinical Advisory Sub-Agent	CRITICAL	The Clinical Advisory Sub-Agent receives Clinical Query / Context messages from the LLM Agent Orchestrator via JSON-RPC without per-message sender attestation. A compromised or rogue Application Zone process can inject crafted clinical queries impersonating the Orchestrator, causing the sub-agent to process unauthorized requests and return manipulated clinical summaries that enter the Orchestrator's response path.
T-9	Clinical Advisory Sub-Agent	CRITICAL	The Clinical Advisory Sub-Agent's context window can be tampered with via two paths: (1) adversarial documents injected into the Knowledge Base that are retrieved during vector search, populating the sub-agent's reasoning context with malicious clinical "facts"; (2) tampering with the Clinical Query / Context payload from the Orchestrator, which may embed attacker-controlled clinical framing. Either path causes the sub-agent to incorporate adversarial content into clinical summaries returned to the Orchestrator.
I-9	Clinical Advisory Sub-Agent	CRITICAL	The Clinical Advisory Sub-Agent processes clinical query context from the Orchestrator and retrieves documents from the Knowledge Base. If its Clinical Summary + Recommendations output is not scrubbed before inclusion in the Orchestrator's HTTPS response to the User, clinical context (patient-specific data, sensitive medical records, proprietary clinical protocols) can leak to unauthorized parties. Additionally, if Clinical Decision Log Entries are not field-classified before writing to the Audit Logger, sensitive clinical data propagates into the training signal stream.
E-7	Clinical Advisory Sub-Agent	CRITICAL	The Clinical Advisory Sub-Agent operates with access to the Knowledge Base and produces clinical outputs that feed directly into the Orchestrator's response path. A prompt injection attack embedded in the Clinical Query / Context payload (or via adversarial KB documents retrieved during

			context assembly) can cause the sub-agent to self-authorize elevated access: issuing requests for documents outside the session's clinical scope, fabricating clinical authority beyond the sub-agent's intended advisory role, or returning outputs designed to manipulate the Orchestrator into taking high-privilege actions (such as invoking high-risk tools).
LLM-13	Clinical Advisory Sub-Agent	CRITICAL	Prompt injection via clinical query context: the Clinical Advisory Sub-Agent processes Clinical Query / Context payloads from the Orchestrator. If the clinical context contains adversarially crafted text (injected via the original user prompt, via adversarial KB documents retrieved upstream, or via a compromised Orchestrator), the injection can override the sub-agent's system prompt, cause it to fabricate clinical recommendations, reveal its system configuration, or escalate privileges within the advisory pipeline.
LLM-14	Clinical Advisory Sub-Agent	CRITICAL	Training data poisoning of the Clinical Advisory Sub-Agent via the Learning Loop: Clinical Decision Log Entries from the sub-agent are included in the Audit Logger training stream. An attacker who injects adversarially crafted clinical interaction records into the Audit Logger can shift the sub-agent's clinical reasoning toward attacker-preferred outputs — for example, consistently recommending specific drugs, understating contraindications, or omitting standard-of-care steps in returned clinical summaries.
MI-1	Clinical Advisory Sub-Agent	CRITICAL	Ungrounded Factual Emission (Category 1 per FR-017): The Clinical Advisory Sub-Agent emits clinical summaries containing factual medical claims (diagnostic observations, drug-interaction assertions, clinical recommendations) to the Orchestrator's response path without mandatory RAG grounding against a verified EHR index. Although the sub-agent retrieves documents from the Knowledge Base via vector search, there is no declared retrieval-strength metric (hit-rate or recall@k), no per-claim source anchoring that traces each clinical assertion to a specific retrieved document section, and no output-time verification that factual claims are supported by retrieved content. A hallucinated clinical assertion (fabricated drug dose, fabricated contraindication, fabricated diagnostic criterion) that reaches a clinician or downstream decision system under time pressure may drive a clinical action the recipient would not otherwise take.
MI-2	Clinical Advisory Sub-Agent	CRITICAL	Overreliance / Missing HITL on Decision-Critical Output (Category 3 per FR-017): The Clinical Advisory Sub-Agent's "Clinical Summary + Recommendations" output flows directly back to the LLM Agent Orchestrator and from there into the user-facing response path without a declared human-in-the-loop (HITL) review gate. Clinical recommendations generated by the sub-agent — including potential drug choices, contraindication assessments, and diagnostic interpretations — surface to the end consumer (clinician, patient, downstream system) as Orchestrator-mediated output without requiring physician sign-off, clinical review confirmation, or AI-provenance disclosure.
MI-3	Clinical Advisory Sub-Agent	CRITICAL	Retrieval-Grounding Gap (Category 4 per FR-017): The Clinical Advisory Sub-Agent performs vector search against the Knowledge Base to retrieve supporting clinical documents. However, there is no declared mechanism to detect or handle retrieval failures — scenarios where the Knowledge Base does not contain documents relevant to the clinical query (low-recall retrieval, out-of-distribution queries, stale KB content). In these cases, the sub-agent may fabricate plausible-sounding clinical content to fill the gap,

			presenting hallucinated information with the same confidence as retrieval-grounded claims.
R-9	Clinical Advisory Sub-Agent	HIGH	The Clinical Advisory Sub-Agent denies having generated a specific clinical summary or recommendation. Without non-repudiable logs of each clinical output (with content hash and the KB documents retrieved to produce it), clinical decisions influenced by sub-agent outputs cannot be attributed, and the sub-agent cannot be held accountable for hallucinated or incorrect recommendations.
D-9	Clinical Advisory Sub-Agent	HIGH	The Clinical Advisory Sub-Agent is invoked by the Orchestrator for each clinical query via JSON-RPC and performs a vector search against the Knowledge Base. High-volume or adversarially-crafted clinical queries (e.g., embedding maximally complex clinical contexts or triggering exhaustive KB retrievals) can exhaust the sub-agent's inference capacity or starve the Knowledge Base of query capacity, disrupting both clinical advisory and baseline Orchestrator retrieval operations.
OI-4	Clinical Advisory Sub-Agent	HIGH	Improper output handling — server-side execution via Clinical Query / Context injection into the Orchestrator's downstream Tool Call Request: the Clinical Advisory Sub-Agent returns "Clinical Summary + Recommendations" to the Orchestrator via JSON-RPC. If the Orchestrator incorporates this clinical output into a subsequent Tool Call Request (JSON-RPC) to the MCP Tool Server without sanitization (e.g., embedding clinical recommendation text into a tool parameter), adversarial content injected into the clinical output can achieve server-side execution at the Tool Server. Execution context: server-side via the Orchestrator→ToolServer path. FR-011 two-part gate confirmed: (1) LLM keyword on Clinical Advisory Sub-Agent ("clinical", "advisory", "medical"), (2) `Clinical Summary + Recommendations` → Orchestrator → `Tool Call Request (JSON-RPC)` → MCP Tool Server = server-side execution sink downstream of sub-agent output.
R-1	User	MEDIUM	A user denies having submitted a particular prompt or query, claiming the record in the Audit Logger was falsified. Without request signing or non-repudiation controls at the User→Guardrails boundary, the system cannot prove the user submitted a specific input.

Unclassified — Unclassified (29 findings)

ID	Component	Severity	Threat
S-5	Inter-Agent Communication Channel	CRITICAL	The Channel is a shared message routing substrate with no inherent sender authentication. A malicious process in the Application Zone can inject messages impersonating either the Orchestrator or the Specialist Agent, enabling unauthorized task injection or result fabrication.
S-7	Long-Running Learning Loop	CRITICAL	The Learning Loop accepts a Training Signal Stream from the Audit Logger without verifying the data source's integrity or authenticity. An attacker who compromises the Audit Logger can inject fabricated training signals, silently manipulating future model updates under the appearance of legitimate operational data.
T-3	Specialist Agent	CRITICAL	The Specialist Agent's operational context can be tampered with by injecting adversarial content into the Delegated Task message via the Inter-Agent Communication Channel. A

			compromised message in the Channel can redirect the Specialist's actions, modify its tool call targets, or exfiltrate data via a fabricated task payload.
T-4	Inter-Agent Communication Channel	CRITICAL	Messages transiting the Inter-Agent Communication Channel can be modified in transit by a process with access to the channel's message queue or shared memory. An agent-in-the-middle attack modifies delegation messages before delivery, redirecting specialist tasks or injecting malicious instructions without detection.
T-8	Long-Running Learning Loop	CRITICAL	The training signal stream from the Audit Logger to the Learning Loop can be poisoned (data poisoning attack) by injecting adversarial entries into the Audit Logger before training runs. A time-delayed attack (temporal attack pattern) inserts adversarial training signals that activate only when a specific trigger pattern appears in future user prompts — a sleeper-agent injection via the model update cycle.
I-4	Inter-Agent Communication Channel	CRITICAL	Messages on the Inter-Agent Communication Channel are observable by any process in the Application Zone with access to the shared message bus or queue. Unencrypted or insufficiently access-controlled inter-agent messages expose sensitive task context to unauthorized observers.
E-4	Inter-Agent Communication Channel	CRITICAL	If the Channel does not enforce sender authentication, any Application Zone process can inject messages with forged identity headers claiming elevated sender roles (e.g., claiming to be the Orchestrator to issue trusted delegation messages). This elevates the attacker from a low-privilege process to the Orchestrator's trust level.
E-6	Long-Running Learning Loop	CRITICAL	The Learning Loop applies model updates with access to the Orchestrator, Specialist Agent, and Clinical Advisory Sub-Agent model parameters. If the update mechanism is compromised (poisoned training data, unauthenticated update channel), an attacker elevates from data-layer access to model-parameter control, effectively gaining the ability to inject arbitrary behaviors into all three agents via the next update cycle.
AG-3	Specialist Agent	CRITICAL	The Specialist Agent, once delegated a task, operates autonomously without continuous Orchestrator oversight. An adversarially crafted delegation message can cause the Specialist to execute a sequence of tool calls that constitutes a prohibited action when viewed holistically — each individual call appears permitted, but the combination achieves an unauthorized outcome.
AG-4	Inter-Agent Communication Channel	CRITICAL	The Channel is a shared substrate whose compromise enables agent-in-the-middle attacks: an attacker intercepts delegation messages, modifies the task parameters (replacing legitimate tool targets with attacker-controlled endpoints), and forwards the modified message to the Specialist Agent. The Specialist executes unauthorized actions believing the instructions came from the Orchestrator.
AG-7	Long-Running Learning Loop	CRITICAL	The Learning Loop's model update mechanism, when fed adversarially crafted training signals, can be exploited for a temporal autonomy attack: the training data contains instructions that cause the updated model to expand its autonomous action scope on the next cycle, gradually accumulating capabilities it was not originally authorized to have.
AG-8	Inter-Agent Communication Channel	CRITICAL	Insecure Inter-Agent Communication (Category 9 — OWASP ASI07:2026): The Inter-Agent Communication Channel connects the LLM Agent Orchestrator, Specialist

			Agent, and (via the Orchestrator) the Clinical Advisory Sub-Agent without declaring mutual authentication, inter-agent message signing, or nonce-based replay prevention. The channel does not declare mTLS between senders and receivers, messages lack HMAC envelope signatures or asymmetric envelope signatures (Ed25519 / ECDSA), and no timestamp-bound nonce-based replay-window enforcement is documented. A network-positioned attacker or a compromised Application Zone process can intercept delegation messages (AML.T0060 agent-in-the-middle topology) and replay, modify, or inject instructions to the Specialist Agent or the Orchestrator without any authentic-source signal available to the receiving component. The Orchestrator additionally acts as a relay between the Channel and the Clinical Advisory Sub-Agent without declared taint propagation — the relay's outputs do not carry the upstream sender's authority labels, enabling an attacker who compromises the Orchestrator's relay function to propagate attacker-controlled content to ClinAdvisor without the authority label that ClinAdvisor would need to detect tampering.
LLM-8	Specialist Agent	CRITICAL	Prompt injection via delegation messages: the Specialist Agent processes tasks delegated by the Orchestrator via the Inter-Agent Channel. An attacker who injects adversarial content into the Delegation Message (via channel tampering or Orchestrator compromise) can hijack the Specialist's task execution, causing it to perform unauthorized tool invocations or exfiltrate data through its result channel.
LLM-9	Specialist Agent	CRITICAL	Training data poisoning of the Specialist Agent via the Learning Loop: adversarially crafted audit log entries from the Specialist's own decision logs (self-poisoning) can be incorporated into the Learning Loop's training signal and returned as a model update that shifts the Specialist's behavior toward attacker-preferred outputs.
LLM-11	Long-Running Learning Loop	CRITICAL	Data poisoning of the Learning Loop's training signal: the audit log training stream is the Learning Loop's primary data source. An attacker who systematically injects adversarially crafted interaction records into the Audit Logger creates poisoned training data that shifts the updated models' behavior over the training cycle — a temporal data poisoning attack with delayed activation at the next model update.
S-4	Specialist Agent	HIGH	The Specialist Agent impersonates the Orchestrator when returning results to the Inter-Agent Communication Channel. A compromised Specialist Agent could inject fabricated "Aggregated Results" back to the Orchestrator, making unauthorized actions appear to have originated from valid specialist work.
S-8	External API	HIGH	The External API provider's identity is not verified beyond TLS certificate validation. An attacker performing DNS hijacking or a BGP route hijack can redirect the MCP Tool Server's outbound API calls to an attacker-controlled server that returns malicious tool results.
R-4	Specialist Agent	HIGH	The Specialist Agent denies having executed a tool call or produced a specific result. Without signed, non-repudiable decision logs, the Specialist's actions cannot be attributed.
R-7	Long-Running Learning Loop	HIGH	The Learning Loop denies having applied a specific model update or claims that an update was applied with different training data than what is recorded. Without cryptographic provenance, model updates cannot be attributed to specific training runs or data sources.

I-3	Specialist Agent	HIGH	The Specialist Agent receives sensitive data via delegated tasks (from the Orchestrator's context) and may include it verbatim in its results returned via the Inter-Agent Channel. If the Channel is observable or the results are logged without redaction, sensitive upstream data leaks downstream.
I-8	Long-Running Learning Loop	HIGH	The Learning Loop consumes the full Audit Logger training signal stream, which includes user prompts, agent decisions, and tool call parameters. If the trained model memorizes sensitive training data, it can inadvertently reproduce PII or proprietary information in its responses (training data extraction attack).
D-3	Specialist Agent	HIGH	The Specialist Agent is invoked by the Orchestrator via the Inter-Agent Channel. An adversarially crafted delegation message that triggers computationally expensive subtasks can exhaust the Specialist Agent's processing capacity, preventing it from completing legitimate delegated work.
D-4	Inter-Agent Communication Channel	HIGH	The Channel's message queue can be flooded by a compromised agent or a malfunctioning process, causing legitimate messages to be dropped, delayed, or rejected. Queue saturation disrupts coordination between Orchestrator and Specialist Agent.
E-3	Specialist Agent	HIGH	The Specialist Agent receives delegated permissions from the Orchestrator. If the delegation message is forged or tampered with (granting the Specialist elevated permissions beyond the original user session scope), the Specialist gains unauthorized capability to invoke tools or access data outside its permitted scope.
LLM-10	Specialist Agent	HIGH	Improper output handling — server-side injection via tool call results: the Specialist Agent's tool call results from MCP Tool Server are incorporated into its context and may influence downstream tool invocations. If the Tool Server returns LLM-influenced content that contains injection payloads, the Specialist's next tool call may forward those payloads to execution sinks. Execution context is server-side.
LLM-12	Long-Running Learning Loop	HIGH	Model theft via Learning Loop output monitoring: an attacker with observability access to the Learning Loop's model update artifacts (parameter diffs, update packages) can reconstruct the model's architecture, parameters, or training data characteristics — effectively stealing the proprietary model.
D-8	Long-Running Learning Loop	MEDIUM	The Learning Loop is a resource-intensive batch process. A high-volume data injection into the Audit Logger (training signal flooding) can cause the Learning Loop to enter runaway processing, consuming excessive compute resources and either blocking legitimate model updates or degrading system performance.
R-5	Inter-Agent Communication Channel	LOW	The Channel denies having delivered or modified a specific message. Without delivery receipts and message integrity records, it is impossible to determine whether a message was delivered as sent or was modified in transit.
R-8	External API	LOW	The External API provider denies having returned a specific response to the MCP Tool Server, enabling disputes over what data was received and acted upon.

MAESTRO Layer — MAESTRO Layer (2 findings)

ID	Component	Severity	Threat
----	-----------	----------	--------

ID	Component	RISK LEVEL	Threat
ID	Component	RISK LEVEL	Threat

87 findings across 9 MAESTRO layers | Sorted by severity (critical first)

Detailed Findings

Findings Detail — Residual Risk

Findings Detail — Residual Risk

HIGH S-1

User

An attacker impersonates a legitimate user by replaying stolen session tokens or forging identity credentials

Implement multi-factor authentication and session token binding at the User→Guardrails boundary. Deploy short-lived JWTs with rotation, session binding to device fingerprint, and server-side token revocation. Add brute-force protection with account lockout and CAPTCHA on repeated failures.

Residual: 8.2 | Control: No Control Found

HIGH AG-1

LLM Agent Orchestrator

Prompt injection causes the Orchestrator to autonomously execute unauthorized high-impact actions

Implement agentic control guardrails: (a) structured output schema validation that rejects Orchestrator actions not conforming to approved schemas, (b) a minimal-footprint enforcement layer capping tool invocations per session, and (c) a human-in-the-loop confirmation gate for high-impact actions (bulk KB queries, External API invocations exceeding defined thresholds).

Residual: 7.8 | Control: No Control Found

HIGH E-2

LLM Agent Orchestrator

The Orchestrator has privileged access to KB, MCP Tool Server, and delegation authority — self-authorization via prompt injection

Implement least-privilege access control for the Orchestrator: (a) scope KB access to query-relevant documents only (not full-corpus access), (b) restrict tool invocation to a pre-approved allowlist per request context, and (c) implement delegation authority limits so the Orchestrator can only delegate tasks within the scope of the original user request. Use ABAC policy evaluation before each privileged operation.

Residual: 7.8 | Control: No Control Found

HIGH E-1

Guardrails Service

Prompt injection that bypasses the Guardrails Service elevates attacker privilege to trusted caller

Harden the Guardrails Service against prompt injection bypass with layered defense: (a) multi-model ensemble filtering (secondary LLM judge evaluating primary filter output), (b) structured output enforcement (Guardrails output is a typed schema, not a raw passthrough), (c) semantic similarity checks against known injection pattern libraries, and (d) rate limiting on re-submission after rejection.

Residual: 7.7 | Control: No Control Found

HIGH D-10

LLM Agent Orchestrator

LLM Inference-Request Flooding and Token Exhaustion without per-tenant QPS rate limiting

Implement per-tenant QPS (queries-per-second) rate limiting at the API gateway layer for the LLM inference endpoint. Enforce max-token-per-request limits to prevent individual requests from consuming the full inference compute budget. Add fan-out budget enforcement that caps the total number of concurrent downstream inference calls (Orchestrator + Specialist + ClinAdvisor) triggered by a single user request. Set automated circuit-breaker thresholds that reject incoming inference requests when the inference queue depth exceeds a configured ceiling.

Residual: 7.2 | Control: No Control Found

HIGH D-11

LLM Agent Orchestrator

Context-Window Exhaustion — Latency-Driven Variant without max-context-window enforcement

Implement max-context-window enforcement at the API gateway before inference requests are dispatched. Reject or truncate conversation history payloads that exceed the declared context limit for the deployed model tier. Add recursive-prompt detection that identifies adversarially constructed long prompts (repeated patterns, padding sequences) and short-circuits them before they enter the inference queue. Apply the same fan-out slot protection as D-10 to prevent a single max-context request from blocking all three inference endpoints simultaneously.

Residual: 7.2 | Control: No Control Found

HIGH I-2

LLM Agent Orchestrator

The Orchestrator's context window contains sensitive data exposed via inference side-channels

Implement context isolation controls to prevent context window leakage: (a) enforce strict system-prompt confidentiality (system prompt never echoed in responses), (b) implement output filtering to detect and redact context-referencing disclosures before sending responses to users, and (c) segment the context window so KB documents and tool results are in separate memory regions not directly accessible to response generation without explicit retrieval.

Residual: 7.2 | Control: No Control Found

HIGH T-2

LLM Agent Orchestrator

The Orchestrator's context window (system prompt) can be tampered with to alter behavior

Implement context window integrity controls: (a) sign all system-prompt segments with HMAC before inserting them into the context window, (b) validate HMAC signatures at context assembly time to detect tampering, and (c) implement content-hash verification on Knowledge Base documents retrieved into context to detect upstream poisoning.

Residual: 7.1 | Control: No Control Found

HIGH E-5

MCP Tool Server

The MCP Tool Server executes tools with credentials of the requesting agent without scope restriction

Implement zero-trust authorization at the MCP Tool Server: each tool call request must carry a signed capability token scoped to the specific tool and parameters; the Tool Server validates the token and rejects calls exceeding the granted scope. Implement tool-call audit logging for all executions, and enforce credential least-privilege so each tool uses its own minimum-privilege service account rather than shared credentials.

Residual: 7.0 | Control: No Control Found

HIGH T-5

MCP Tool Server

Tool call request parameters supplied by agents can be tampered with to achieve injection

Implement strict tool call parameter validation at the MCP Tool Server: define a JSON Schema for each registered tool's `params` object and validate all incoming parameters against it before dispatch. Reject calls where parameters fail schema validation. Implement parameter sanitization stripping SQL injection, shell metacharacters, and path traversal sequences from string parameters.

Residual: 7.0 | Control: No Control Found

MEDIUM R-3

LLM Agent Orchestrator

The Orchestrator denies having issued a specific delegation message or tool call request

|

Residual: 5.9 | Control: Partial Control

MEDIUM LLM-6

LLM Agent Orchestrator

Improper output handling — server-side execution via Tool Call Request

Extend the existing Guardrails input validation to cover LLM-synthesized JSON-RPC parameters flowing to the MCP Tool Server. Implement a tool parameter sanitization layer at the Tool Server ingress that validates each parameter against the tool's declared JSON Schema and strips injection payloads (SQL fragments, shell metacharacters, template expressions) before dispatch.

Residual: 5.8 | Control: Partial Control

MEDIUM **OI-2**

LLM Agent Orchestrator

Improper output handling — server-side execution via Tool Call Request (OI category)

Implement a dedicated output sanitization layer treating all LLM-generated content destined for server-side execution sinks as untrusted input. Validate parameter schemas, strip injection metacharacters, and require parameterized invocations for all database and shell tool calls. This is the OI-category complement to LLM-6 and should be implemented together.

Residual: 5.8 | Control: Partial Control

MEDIUM **LLM-5**

LLM Agent Orchestrator

Improper output handling — client-side XSS via LLM-generated response content

Implement client-side XSS prevention: (a) apply HTML output encoding to all LLM-generated content before client rendering, (b) deploy a Content Security Policy (CSP) header restricting script execution to approved sources, and (c) use a DOM sanitization library to strip executable markup from LLM responses. The architecture passes LLM output directly to the User without client-side sanitization.

Residual: 5.6 | Control: Partial Control

MEDIUM **OI-1**

LLM Agent Orchestrator

Improper output handling — client-side XSS via LLM-generated response content (OI category)

OI-1 covers the same XSS attack vector as LLM-5 from the Output Integrity taxonomy perspective. Implementing the LLM-5 remediation (output encoding + CSP + DOM sanitization) fully addresses OI-1 with no additional changes required.

Residual: 5.6 | Control: Partial Control

MEDIUM **LLM-13**

Clinical Advisory Sub-Agent

Prompt injection via clinical query context injected into the Sub-Agent

Extend the logging-based partial control to include clinical query input validation: (a) implement a clinical context schema validator enforcing structured clinical query formats and rejecting free-form injection attempts, (b) add a prompt integrity check detecting instruction-override patterns in clinical context messages, and (c) require HMAC-signed clinical context objects from the Orchestrator to the Sub-Agent to prevent in-transit tampering.

Residual: 5.6 | Control: Partial Control

MEDIUM LLM-8

Specialist Agent

Prompt injection via delegation messages from the Inter-Agent Communication Channel

Extend the existing Guardrails input validation to cover the delegation message path: implement a delegation message schema validator that enforces structured task specifications and rejects free-form prompt-injection patterns in task descriptions. Add HMAC signing of delegation messages from the Orchestrator and verify signatures at the Specialist Agent ingress before task execution.

Residual: 5.5 | Control: Partial Control

MEDIUM LLM-1

LLM Agent Orchestrator

Direct prompt injection via the User→Guardrails boundary reaching the Orchestrator

Harden the existing Guardrails input validation to improve direct prompt injection resistance: (a) implement adversarial prompt detection using a secondary classifier, (b) enforce structured prompt templates that constrain user-controllable text regions, and (c) add jailbreak pattern matching against a curated injection pattern library.

Residual: 5.4 | Control: Partial Control

MEDIUM LLM-15

LLM Agent Orchestrator

Cost Amplification via Recursive or Cost-Asymmetric Prompting without output-token caps

Implement output-amplification monitoring and cost-per-query alerting: (a) enforce per-request output-token caps at the LLM API client level before dispatching to the model provider, (b) track cumulative token consumption per session across all fan-out legs (Orchestrator + Specialist + ClinAdvisor), (c) emit cost-per-query metrics to the monitoring stack with threshold-based alerts when a single request exceeds a cost ceiling, and (d) implement recursive prompt depth tracking that aborts multi-hop chains exceeding a configured depth limit.

Residual: 5.4 | Control: Partial Control

MEDIUM LLM-16

LLM Agent Orchestrator

Denial-of-Wallet via Context-Window Cost Amplification without per-tenant token budget hard-cap

Implement per-tenant token budget hard-caps with automated tenant suspension on budget exhaustion: (a) enforce daily/monthly token budgets per tenant at the API gateway, rejecting requests that would exceed the tenant's remaining budget, (b) implement denial-of-wallet anomaly detection comparing current spend rate to tenant baseline and triggering alerts on anomalous acceleration, (c) add automated tenant suspension that pauses a tenant's access when spend exceeds a 2x daily baseline within any rolling 1-hour window.

Residual: 5.4 | Control: Partial Control

MEDIUM LLM-4

LLM Agent Orchestrator

Training data poisoning via the Long-Running Learning Loop model update cycle

Extend the existing audit logging to include training data provenance tracking: log the content hash of each audit entry contributing to the training signal stream, and implement a validation step in the Learning Loop that verifies audit entry hashes before incorporating them into training. This makes poisoned audit entries detectable through hash chain breaks.

Residual: 5.3 | Control: Partial Control

MEDIUM E-7

Clinical Advisory Sub-Agent

The Clinical Advisory Sub-Agent operates with elevated access to sensitive clinical data

Implement least-privilege access control for the Clinical Advisory Sub-Agent: restrict its KB retrieval to clinical document namespaces only, enforce read-only access to the KB, and implement capability tokens limiting the Sub-Agent to clinical summary operations. Apply ABAC policy evaluation at each KB retrieval call.

Residual: 6.9 | Control: No Control Found

MEDIUM AG-5

MCP Tool Server

The MCP Tool Server is vulnerable to tool call parameter injection

Residual: 5.2 | Control: Partial Control

MEDIUM I-9

Clinical Advisory Sub-Agent

The Clinical Advisory Sub-Agent processes clinical data without encryption or minimization

Implement encryption and minimization for clinical data processed by the Clinical Advisory Sub-Agent: (a) encrypt clinical query payloads in transit between the Orchestrator and the Sub-Agent using TLS 1.3 mutual authentication, (b) implement field-level data minimization so the Sub-Agent receives only the clinical fields required for each query, and (c) enforce PII masking on clinical data before logging to the Audit Logger.

Residual: 6.8 | Control: No Control Found

MEDIUM LLM-2

LLM Agent Orchestrator

Indirect prompt injection via the Knowledge Base retrieval path

Extend the existing Guardrails input validation to the Knowledge Base retrieval path: implement document content scanning at KB retrieval time that detects adversarially injected prompt override patterns in retrieved documents before they enter the Orchestrator's context window. Add retrieval-time content hash verification against a trusted document registry.

Residual: 5.1 | Control: Partial Control

MEDIUM **D-1**

Guardrails Service

The Guardrails Service is vulnerable to resource exhaustion / denial-of-service

■

Residual: 5.0 | Control: Partial Control

MEDIUM **LLM-7**

LLM Agent Orchestrator

Improper output handling — SSRF via LLM-synthesized URL in tool call

■

Residual: 5.0 | Control: Partial Control

MEDIUM **LLM-14**

Clinical Advisory Sub-Agent

Training data poisoning of the Clinical Advisory Sub-Agent via poisoned KB documents

■

Residual: 5.0 | Control: Partial Control

MEDIUM **MI-2**

Clinical Advisory Sub-Agent

Overreliance / Missing HITL on Decision-Critical clinical outputs

■

Residual: 6.6 | Control: No Control Found

MEDIUM **T-9**

Clinical Advisory Sub-Agent

The Clinical Advisory Sub-Agent's context window can be tampered with via poisoned clinical context

■

Residual: 6.6 | Control: No Control Found

MEDIUM **LLM-10**

Specialist Agent

Improper output handling — server-side injection via Specialist Agent tool call

■

Residual: 4.9 | Control: Partial Control

MEDIUM **MI-1**

Clinical Advisory Sub-Agent

Ungrounded Factual Emission — clinical claims without source attribution

■

Residual: 6.5 | Control: No Control Found

MEDIUM **MI-3**

Clinical Advisory Sub-Agent

Retrieval-Grounding Gap — clinical summaries not entailed by retrieved documents

■

Residual: 6.5 | Control: No Control Found

MEDIUM **OI-3**

LLM Agent Orchestrator

Improper output handling — SSRF via LLM-synthesized URL (OI category)

■

Residual: 4.9 | Control: Partial Control

MEDIUM **E-3**

Specialist Agent

The Specialist Agent receives delegated permissions beyond necessary scope

■

Residual: 6.4 | Control: No Control Found

MEDIUM **E-4**

Inter-Agent Communication Channel

The Channel does not enforce sender authentication — privilege escalation via impersonation

■

Residual: 6.4 | Control: No Control Found

MEDIUM **S-5**

Inter-Agent Communication Channel

The Channel is a shared message routing substrate vulnerable to spoofing

■

Residual: 6.4 | Control: No Control Found

MEDIUM S-7

Long-Running Learning Loop

The Learning Loop accepts a Training Signal Stream without sender verification

■

Residual: 6.4 | Control: No Control Found

MEDIUM AG-3

Specialist Agent

The Specialist Agent, once delegated a task, operates beyond its intended scope

■

Residual: 4.7 | Control: Partial Control

MEDIUM AG-6

MCP Tool Server

The MCP Tool Server acts as a privileged execution environment without action constraints

■

Residual: 4.7 | Control: Partial Control

MEDIUM D-2

LLM Agent Orchestrator

The Orchestrator's inference pipeline is a bounded resource vulnerable to token flooding

■

Residual: 6.2 | Control: No Control Found

MEDIUM D-5

MCP Tool Server

The Tool Server's capacity for concurrent External API calls can be exhausted

■

Residual: 4.7 | Control: Partial Control

MEDIUM OI-4

Clinical Advisory Sub-Agent

Improper output handling — server-side execution via Clinical Advisory output

■

Residual: 4.7 | Control: Partial Control

MEDIUM R-1

User

A user denies having submitted a particular prompt (non-repudiation gap)

|

Residual: 4.7 | Control: Partial Control

MEDIUM S-6

MCP Tool Server

An attacker in the Application Zone spoofs a valid MCP tool caller identity

|

Residual: 6.2 | Control: No Control Found

MEDIUM AG-2

LLM Agent Orchestrator

The Orchestrator and Specialist Agent can jointly exhibit emergent cascading behavior

|

Residual: 6.1 | Control: No Control Found

MEDIUM AG-4

Inter-Agent Communication Channel

The Channel is a shared substrate whose compromise enables cascading failures

|

Residual: 6.0 | Control: No Control Found

MEDIUM E-6

Long-Running Learning Loop

The Learning Loop applies model updates without cryptographic verification

|

Residual: 6.0 | Control: No Control Found

MEDIUM I-1

Guardrails Service

The Guardrails Service leaks rejected prompt content in error responses

|

Residual: 5.6 | Control: No Control Found

MEDIUM S-3

LLM Agent Orchestrator

The Orchestrator's identity is not cryptographically bound — impersonation risk

■

Residual: 5.9 | Control: No Control Found

MEDIUM T-3

Specialist Agent

The Specialist Agent's operational context can be tampered with via delegation messages

■

Residual: 5.9 | Control: No Control Found

MEDIUM T-4

Inter-Agent Communication Channel

Messages transiting the Channel lack integrity protection — in-transit tampering

■

Residual: 5.9 | Control: No Control Found

MEDIUM S-8

External API

The External API provider's identity is not verified by the MCP Tool Server

■

Residual: 5.8 | Control: No Control Found

MEDIUM S-9

Clinical Advisory Sub-Agent

The Clinical Advisory Sub-Agent receives Clinical Query / Context without sender authentication

■

Residual: 5.8 | Control: No Control Found

MEDIUM T-7

Audit Logger

The Audit Logger entries can be tampered with by any component with write access

■

Residual: 5.8 | Control: No Control Found

MEDIUM AG-7

Long-Running Learning Loop

The Learning Loop's model update mechanism is vulnerable to temporal attacks

I

Residual: 5.6 | Control: No Control Found

MEDIUM D-9

Clinical Advisory Sub-Agent

The Clinical Advisory Sub-Agent is invoked by the Orchestrator and its inference capacity is bounded

I

Residual: 4.2 | Control: Partial Control

MEDIUM LLM-9

Specialist Agent

Training data poisoning of the Specialist Agent's learned behavior via audit log

I

Residual: 4.2 | Control: Partial Control

MEDIUM D-6

Knowledge Base

The Knowledge Base can be rendered unavailable through targeted query flooding

I

Residual: 5.7 | Control: No Control Found

MEDIUM LLM-11

Long-Running Learning Loop

Data poisoning of the Learning Loop's training signal via audit log injection

I

Residual: 5.7 | Control: No Control Found

MEDIUM T-8

Long-Running Learning Loop

Training signal stream from Audit Logger to Learning Loop can be poisoned (temporal attack)

I

Residual: 5.7 | Control: No Control Found

MEDIUM D-3

Specialist Agent

The Specialist Agent is invoked by the Orchestrator and its inference capacity is bounded

■

Residual: 4.1 | Control: Partial Control

MEDIUM D-4

Inter-Agent Communication Channel

The Channel's message queue can be flooded by a high-volume sender

■

Residual: 5.5 | Control: No Control Found

MEDIUM D-7

Audit Logger

The Audit Logger can be overwhelmed by a log-flooding attack

■

Residual: 5.5 | Control: No Control Found

MEDIUM AG-8

Inter-Agent Communication Channel

Insecure Inter-Agent Communication — lack of authentication and integrity controls

■

Residual: 5.5 | Control: No Control Found

MEDIUM I-6

Knowledge Base

The Knowledge Base exposes its full document corpus to any component with retrieval access

■

Residual: 5.4 | Control: No Control Found

MEDIUM I-7

Audit Logger

The Audit Logger aggregates sensitive data from all pipeline components without access control

■

Residual: 5.4 | Control: No Control Found

MEDIUM D-8

Long-Running Learning Loop

The Learning Loop is a resource-intensive batch process vulnerable to resource competition

I

Residual: 5.3 | Control: No Control Found

MEDIUM I-5

MCP Tool Server

Tool results from External API calls may contain sensitive data passed through without masking

I

Residual: 4.0 | Control: Partial Control

MEDIUM LLM-3

LLM Agent Orchestrator

Model theft via systematic API probing of the Orchestrator

I

Residual: 4.0 | Control: Partial Control

MEDIUM LLM-12

Long-Running Learning Loop

Model theft via Learning Loop output monitoring

I

Residual: 5.3 | Control: No Control Found

MEDIUM T-6

Knowledge Base

The Knowledge Base corpus can be tampered with by components with write access

I

Residual: 5.3 | Control: No Control Found

MEDIUM S-2

Guardrails Service

An attacker spoofs the Guardrails Service by sending crafted validation responses

I

Residual: 5.2 | Control: No Control Found

MEDIUM I-4

Inter-Agent Communication Channel

Messages on the Channel may expose sensitive payloads to eavesdropping components

I

Residual: 5.1 | Control: No Control Found

MEDIUM I-8

Long-Running Learning Loop

The Learning Loop consumes the full Audit Logger training stream without data minimization

I

Residual: 4.9 | Control: No Control Found

MEDIUM R-7

Long-Running Learning Loop

The Learning Loop denies having applied a specific model update

I

Residual: 4.9 | Control: No Control Found

MEDIUM S-4

Specialist Agent

The Specialist Agent impersonates the Orchestrator when responding via the Channel

I

Residual: 4.9 | Control: No Control Found

MEDIUM I-3

Specialist Agent

The Specialist Agent receives sensitive data via delegation messages without encryption

I

Residual: 4.8 | Control: No Control Found

MEDIUM AGP-01

LLM Agent Orchestrator

Multi-agent emergent behavior — cascading failure through delegation chains

I

Residual: 4.6 | Control: No Control Found

MEDIUM R-5	Inter-Agent Communication Channel
The Channel denies having delivered or modified a message	
█	
Residual: 4.3 Control: No Control Found	
LOW R-9	Clinical Advisory Sub-Agent
The Clinical Advisory Sub-Agent denies having generated a specific clinical recommendation	
█	
Residual: 3.9 Control: Partial Control	
LOW R-8	External API
The External API provider denies having returned a specific response	
█	
Residual: 3.8 Control: Partial Control	
LOW R-2	Guardrails Service
The Guardrails Service denies having logged a filtering decision	
█	
Residual: 3.3 Control: Partial Control	
LOW R-4	Specialist Agent
The Specialist Agent denies having executed a delegated task	
█	
Residual: 3.3 Control: Partial Control	
LOW R-6	MCP Tool Server
The MCP Tool Server denies having executed a specific tool call	
█	
Residual: 3.3 Control: Partial Control	

Control Coverage

Control Status by STRIDE Category

STRIDE Category	Found	Partial	Missing	Total
Spoofing	0	0	9	9
Tampering	0	0	8	8
Repudiation	0	7	2	9
Information Disclosure	0	1	8	9
Denial of Service	0	4	7	11
Elevation of Privilege	0	0	7	7
Agentic Threats	0	3	5	8
LLM Threats	0	14	2	16
Other	0	4	4	8

Detailed Control Assessment

Component	Control Category	Status	Evidence	Effectiveness
LLM Agent Orchestrator	Logging/Audit	Partial	architecture.md:47–58	Moderate
LLM Agent Orchestrator	Logging/Audit	Partial	architecture.md:50	Moderate
Clinical Advisory Sub-Agent	Logging/Audit	Partial	architecture.md:58	Moderate
LLM Agent Orchestrator	Input Validation	Partial	architecture.md:31–33	Moderate

Coverage Summary



Control Status	Count	Percentage
Control Found	0	0%
Partial Control	33	37.5%
No Control Found	55	62.5%
Total	88	100%

Remediation Roadmap

■ Critical: 0 ■ High: 10 ■ Medium: 70 ■ Low: 5

85 total actions

Priority	Finding	Recommendation	SLA	Status
HIGH 10 actions				
S-1	An attacker impersonates a legitimate user by replaying stolen session tokens or forging identity credentials	Implement multi-factor authentication and session token binding at the User→Guardrails boundary. Deploy short-lived JWTs with rotation, session binding to device fingerprint, and server-side token revocation. Add brute-force protection with account lockout and CAPTCHA on repeated failures.	14d	No Control Found
AG-1	Prompt injection causes the Orchestrator to autonomously execute unauthorized high-impact actions	Implement agentic control guardrails: (a) structured output schema validation that rejects Orchestrator actions not conforming to approved schemas, (b) a minimal-footprint enforcement layer capping tool invocations per session, and (c) a human-in-the-loop confirmation gate for high-impact actions (bulk KB queries, External API invocations exceeding defined thresholds).	14d	No Control Found
E-2	The Orchestrator has privileged access to KB, MCP Tool Server, and delegation authority — self-authorization via prompt injection	Implement least-privilege access control for the Orchestrator: (a) scope KB access to query-relevant documents only (not full-corpus access), (b) restrict tool invocation to a pre-approved allowlist per request context, and (c) implement delegation authority limits so the Orchestrator can only delegate tasks within the scope of the original user request. Use ABAC policy evaluation before each privileged operation.	14d	No Control Found
E-1	Prompt injection that bypasses the Guardrails Service elevates attacker privilege to trusted caller	Harden the Guardrails Service against prompt injection bypass with layered defense: (a) multi-model ensemble filtering (secondary LLM judge evaluating primary filter output), (b) structured output enforcement (Guardrails output is a typed schema, not a raw passthrough), (c) semantic similarity checks against known injection pattern libraries, and (d) rate limiting on re-submission after rejection.	14d	No Control Found
D-10	LLM Inference-Request Flooding and Token Exhaustion without per-tenant QPS rate limiting	Implement per-tenant QPS (queries-per-second) rate limiting at the API gateway layer for the LLM inference endpoint. Enforce max-token-per-request limits to prevent individual requests from consuming the full inference compute budget. Add fan-	14d	No Control Found

Priority	Finding	Recommendation	SLA	Status
		out budget enforcement that caps the total number of concurrent downstream inference calls (Orchestrator + Specialist + ClinAdvisor) triggered by a single user request. Set automated circuit-breaker thresholds that reject incoming inference requests when the inference queue depth exceeds a configured ceiling.		
D-11	Context-Window Exhaustion — Latency-Driven Variant without max-context-window enforcement	Implement max-context-window enforcement at the API gateway before inference requests are dispatched. Reject or truncate conversation history payloads that exceed the declared context limit for the deployed model tier. Add recursive-prompt detection that identifies adversarially constructed long prompts (repeated patterns, padding sequences) and short-circuits them before they enter the inference queue. Apply the same fan-out slot protection as D-10 to prevent a single max-context request from blocking all three inference endpoints simultaneously.	14d	No Control Found
I-2	The Orchestrator's context window contains sensitive data exposed via inference side-channels	Implement context isolation controls to prevent context window leakage: (a) enforce strict system-prompt confidentiality (system prompt never echoed in responses), (b) implement output filtering to detect and redact context-referencing disclosures before sending responses to users, and (c) segment the context window so KB documents and tool results are in separate memory regions not directly accessible to response generation without explicit retrieval.	14d	No Control Found
T-2	The Orchestrator's context window (system prompt) can be tampered with to alter behavior	Implement context window integrity controls: (a) sign all system-prompt segments with HMAC before inserting them into the context window, (b) validate HMAC signatures at context assembly time to detect tampering, and (c) implement content-hash verification on Knowledge Base documents retrieved into context to detect upstream poisoning.	14d	No Control Found
E-5	The MCP Tool Server executes tools with credentials of the requesting agent without scope restriction	Implement zero-trust authorization at the MCP Tool Server: each tool call request must carry a signed capability token scoped to the specific tool and parameters; the Tool Server validates the token and rejects calls exceeding the granted scope. Implement tool-call audit logging for all executions, and enforce credential least-privilege so each tool uses its own minimum-privilege service account rather than shared credentials.	14d	No Control Found
T-5	Tool call request parameters supplied by	Implement strict tool call parameter validation at the MCP Tool Server: define a	14d	No Control Found

Priority	Finding	Recommendation	SLA	Status
	agents can be tampered with to achieve injection	JSON Schema for each registered tool's `params` object and validate all incoming parameters against it before dispatch. Reject calls where parameters fail schema validation. Implement parameter sanitization stripping SQL injection, shell metacharacters, and path traversal sequences from string parameters.		
MEDIUM 70 actions				
R-3	The Orchestrator denies having issued a specific delegation message or tool call request		30d	Partial Control
LLM-6	Improper output handling — server-side execution via Tool Call Request	Extend the existing Guardrails input validation to cover LLM-synthesized JSON-RPC parameters flowing to the MCP Tool Server. Implement a tool parameter sanitization layer at the Tool Server ingress that validates each parameter against the tool's declared JSON Schema and strips injection payloads (SQL fragments, shell metacharacters, template expressions) before dispatch.	30d	Partial Control
OI-2	Improper output handling — server-side execution via Tool Call Request (OI category)	Implement a dedicated output sanitization layer treating all LLM-generated content destined for server-side execution sinks as untrusted input. Validate parameter schemas, strip injection metacharacters, and require parameterized invocations for all database and shell tool calls. This is the OI-category complement to LLM-6 and should be implemented together.	30d	Partial Control
LLM-5	Improper output handling — client-side XSS via LLM-generated response content	Implement client-side XSS prevention: (a) apply HTML output encoding to all LLM-generated content before client rendering, (b) deploy a Content Security Policy (CSP) header restricting script execution to approved sources, and (c) use a DOM sanitization library to strip executable markup from LLM responses. The architecture passes LLM output directly to the User without client-side sanitization.	30d	Partial Control
OI-1	Improper output handling — client-side XSS via LLM-generated response content (OI category)	OI-1 covers the same XSS attack vector as LLM-5 from the Output Integrity taxonomy perspective. Implementing the LLM-5 remediation (output encoding + CSP + DOM sanitization) fully addresses OI-1 with no additional changes required.	30d	Partial Control
LLM-13	Prompt injection via clinical query context	Extend the logging-based partial control to include clinical query input validation: (a)	30d	Partial Control

Priority	Finding	Recommendation	SLA	Status
	injected into the Sub-Agent	implement a clinical context schema validator enforcing structured clinical query formats and rejecting free-form injection attempts, (b) add a prompt integrity check detecting instruction-override patterns in clinical context messages, and (c) require HMAC-signed clinical context objects from the Orchestrator to the Sub-Agent to prevent in-transit tampering.		
LLM-8	Prompt injection via delegation messages from the Inter-Agent Communication Channel	Extend the existing Guardrails input validation to cover the delegation message path: implement a delegation message schema validator that enforces structured task specifications and rejects free-form prompt-injection patterns in task descriptions. Add HMAC signing of delegation messages from the Orchestrator and verify signatures at the Specialist Agent ingress before task execution.	30d	Partial Control
LLM-1	Direct prompt injection via the User→Guardrails boundary reaching the Orchestrator	Harden the existing Guardrails input validation to improve direct prompt injection resistance: (a) implement adversarial prompt detection using a secondary classifier, (b) enforce structured prompt templates that constrain user-controllable text regions, and (c) add jailbreak pattern matching against a curated injection pattern library.	30d	Partial Control
LLM-15	Cost Amplification via Recursive or Cost-Asymmetric Prompting without output-token caps	Implement output-amplification monitoring and cost-per-query alerting: (a) enforce per-request output-token caps at the LLM API client level before dispatching to the model provider, (b) track cumulative token consumption per session across all fan-out legs (Orchestrator + Specialist + ClinAdvisor), (c) emit cost-per-query metrics to the monitoring stack with threshold-based alerts when a single request exceeds a cost ceiling, and (d) implement recursive prompt depth tracking that aborts multi-hop chains exceeding a configured depth limit.	30d	Partial Control
LLM-16	Denial-of-Wallet via Context-Window Cost Amplification without per-tenant token budget hard-cap	Implement per-tenant token budget hard-caps with automated tenant suspension on budget exhaustion: (a) enforce daily/monthly token budgets per tenant at the API gateway, rejecting requests that would exceed the tenant's remaining budget, (b) implement denial-of-wallet anomaly detection comparing current spend rate to tenant baseline and triggering alerts on anomalous acceleration, (c) add automated tenant suspension that pauses a tenant's access when spend exceeds a 2x daily baseline within any rolling 1-hour window.	30d	Partial Control

Priority	Finding	Recommendation	SLA	Status
LLM-4	Training data poisoning via the Long-Running Learning Loop model update cycle	Extend the existing audit logging to include training data provenance tracking: log the content hash of each audit entry contributing to the training signal stream, and implement a validation step in the Learning Loop that verifies audit entry hashes before incorporating them into training. This makes poisoned audit entries detectable through hash chain breaks.	30d	Partial Control
E-7	The Clinical Advisory Sub-Agent operates with elevated access to sensitive clinical data	Implement least-privilege access control for the Clinical Advisory Sub-Agent: restrict its KB retrieval to clinical document namespaces only, enforce read-only access to the KB, and implement capability tokens limiting the Sub-Agent to clinical summary operations. Apply ABAC policy evaluation at each KB retrieval call.	30d	No Control Found
AG-5	The MCP Tool Server is vulnerable to tool call parameter injection		30d	Partial Control
I-9	The Clinical Advisory Sub-Agent processes clinical data without encryption or minimization	Implement encryption and minimization for clinical data processed by the Clinical Advisory Sub-Agent: (a) encrypt clinical query payloads in transit between the Orchestrator and the Sub-Agent using TLS 1.3 mutual authentication, (b) implement field-level data minimization so the Sub-Agent receives only the clinical fields required for each query, and (c) enforce PII masking on clinical data before logging to the Audit Logger.	30d	No Control Found
LLM-2	Indirect prompt injection via the Knowledge Base retrieval path	Extend the existing Guardrails input validation to the Knowledge Base retrieval path: implement document content scanning at KB retrieval time that detects adversarially injected prompt override patterns in retrieved documents before they enter the Orchestrator's context window. Add retrieval-time content hash verification against a trusted document registry.	30d	Partial Control
D-1	The Guardrails Service is vulnerable to resource exhaustion / denial-of-service		30d	Partial Control
LLM-7	Improper output handling — SSRF via LLM-synthesized URL in tool call		30d	Partial Control
LLM-14	Training data poisoning of the Clinical Advisory Sub-		30d	Partial Control

Priority	Finding	Recommendation	SLA	Status
	Agent via poisoned KB documents			
MI-2	Overreliance / Missing HITL on Decision-Critical clinical outputs		30d	No Control Found
T-9	The Clinical Advisory Sub-Agent's context window can be tampered with via poisoned clinical context		30d	No Control Found
LLM-10	Improper output handling — server-side injection via Specialist Agent tool call		30d	Partial Control
MI-1	Ungrounded Factual Emission — clinical claims without source attribution		30d	No Control Found
MI-3	Retrieval-Grounding Gap — clinical summaries not entailed by retrieved documents		30d	No Control Found
OI-3	Improper output handling — SSRF via LLM-synthesized URL (OI category)		30d	Partial Control
E-3	The Specialist Agent receives delegated permissions beyond necessary scope		30d	No Control Found
E-4	The Channel does not enforce sender authentication — privilege escalation via impersonation		30d	No Control Found
S-5	The Channel is a shared message routing substrate vulnerable to spoofing		30d	No Control Found
S-7	The Learning Loop accepts a Training Signal Stream without sender verification		30d	No Control Found
AG-3	The Specialist Agent, once delegated a task, operates beyond its intended scope		30d	Partial Control
AG-6	The MCP Tool Server acts as a privileged		30d	Partial Control

Priority	Finding	Recommendation	SLA	Status
	execution environment without action constraints			
D-2	The Orchestrator's inference pipeline is a bounded resource vulnerable to token flooding		30d	No Control Found
D-5	The Tool Server's capacity for concurrent External API calls can be exhausted		30d	Partial Control
OI-4	Improper output handling — server-side execution via Clinical Advisory output		30d	Partial Control
R-1	A user denies having submitted a particular prompt (non-repudiation gap)		30d	Partial Control
S-6	An attacker in the Application Zone spoofs a valid MCP tool caller identity		30d	No Control Found
AG-2	The Orchestrator and Specialist Agent can jointly exhibit emergent cascading behavior		30d	No Control Found
AG-4	The Channel is a shared substrate whose compromise enables cascading failures		30d	No Control Found
E-6	The Learning Loop applies model updates without cryptographic verification		30d	No Control Found
I-1	The Guardrails Service leaks rejected prompt content in error responses		30d	No Control Found
S-3	The Orchestrator's identity is not cryptographically bound — impersonation risk		30d	No Control Found
T-3	The Specialist Agent's operational context can be tampered with via delegation messages		30d	No Control Found

Priority	Finding	Recommendation	SLA	Status
T-4	Messages transiting the Channel lack integrity protection — in-transit tampering		30d	No Control Found
S-8	The External API provider's identity is not verified by the MCP Tool Server		30d	No Control Found
S-9	The Clinical Advisory Sub-Agent receives Clinical Query / Context without sender authentication		30d	No Control Found
T-7	The Audit Logger entries can be tampered with by any component with write access		30d	No Control Found
AG-7	The Learning Loop's model update mechanism is vulnerable to temporal attacks		30d	No Control Found
D-9	The Clinical Advisory Sub-Agent is invoked by the Orchestrator and its inference capacity is bounded		30d	Partial Control
LLM-9	Training data poisoning of the Specialist Agent's learned behavior via audit log		30d	Partial Control
D-6	The Knowledge Base can be rendered unavailable through targeted query flooding		30d	No Control Found
LLM-11	Data poisoning of the Learning Loop's training signal via audit log injection		30d	No Control Found
T-8	Training signal stream from Audit Logger to Learning Loop can be poisoned (temporal attack)		30d	No Control Found
D-3	The Specialist Agent is invoked by the Orchestrator and its inference capacity is bounded		30d	Partial Control

Priority	Finding	Recommendation	SLA	Status
D-4	The Channel's message queue can be flooded by a high-volume sender		30d	No Control Found
D-7	The Audit Logger can be overwhelmed by a log-flooding attack		30d	No Control Found
AG-8	Insecure Inter-Agent Communication — lack of authentication and integrity controls		30d	No Control Found
I-6	The Knowledge Base exposes its full document corpus to any component with retrieval access		30d	No Control Found
I-7	The Audit Logger aggregates sensitive data from all pipeline components without access control		30d	No Control Found
D-8	The Learning Loop is a resource-intensive batch process vulnerable to resource competition		30d	No Control Found
I-5	Tool results from External API calls may contain sensitive data passed through without masking		30d	Partial Control
LLM-3	Model theft via systematic API probing of the Orchestrator		30d	Partial Control
LLM-12	Model theft via Learning Loop output monitoring		30d	No Control Found
T-6	The Knowledge Base corpus can be tampered with by components with write access		30d	No Control Found
S-2	An attacker spoofs the Guardrails Service by sending crafted validation responses		30d	No Control Found
I-4	Messages on the Channel may expose sensitive payloads to eavesdropping components		30d	No Control Found

Priority	Finding	Recommendation	SLA	Status
I-8	The Learning Loop consumes the full Audit Logger training stream without data minimization		30d	No Control Found
R-7	The Learning Loop denies having applied a specific model update		30d	No Control Found
S-4	The Specialist Agent impersonates the Orchestrator when responding via the Channel		30d	No Control Found
I-3	The Specialist Agent receives sensitive data via delegation messages without encryption		30d	No Control Found
AGP-01	Multi-agent emergent behavior — cascading failure through delegation chains		30d	No Control Found
R-5	The Channel denies having delivered or modified a message		30d	No Control Found
LOW 5 actions				
R-9	The Clinical Advisory Sub-Agent denies having generated a specific clinical recommendation		90d	Partial Control
R-8	The External API provider denies having returned a specific response		90d	Partial Control
R-2	The Guardrails Service denies having logged a filtering decision		90d	Partial Control
R-4	The Specialist Agent denies having executed a delegated task		90d	Partial Control
R-6	The MCP Tool Server denies having executed a specific tool call		90d	Partial Control